Chapter 2. Building and Running Modules

It's almost time to begin programming. This chapter introduces all the essential concepts about modules and kernel programming. In these few pages, we build and run a complete (if relatively useless) module, and look at some of the basic code shared by all modules. Developing such expertise is an essential foundation for any kind of modularized driver. To avoid throwing in too many concepts at once, this chapter talks only about modules, without referring to any specific device class.

All the kernel items (functions, variables, header files, and macros) that are introduced here are described in a reference section at the end of the chapter.

Setting Up Your Test System

Starting with this chapter, we present exampl e modules to demonstrate programming concepts. (All of these examples are available on O'Reilly's FTP site, as explained in Chapter 1.) Bu ilding, lo ading, and modifying these exa mples are a good way to improve your understanding of how drivers work and interact with the kernel.

The example modules should work with almost any 2.6.x kernel, including those provided by distribution vendors. However, we recommend that you obtain a "mainline" kernel directly from the *kernel.org* mirror network, and install it on your system. Vendor kernels can be heavily patched and divergent from the mainline; at times, vendor patches can change the kernel API as se en by devi ce drivers . If you are writing a driver that must work on a particular distribution, you will certainly want to build and test against the relevant kernels. But, for the purpose of learning about driver writing, a standard kernel is best.

Regardless of the origin of your kernel, building modules for 2.6.x requires that you have a configured and built kernel tree on your system. This requirement is a change from previous versions of the kernel, where a current set of header files was sufficient. 2.6 modules are linked against object files found in the kernel source tree; the result is a more robust module loader, but also the requirement that those object files be available. So your first order of business is to come up with a kernel source tree (either from the *kernel.org* network or your distributor's kernel source package), build a new kernel, and install it on your system. For reasons we'll see later, life is generally easiest if you are actually running the target kernel when you build your modules, though this is not required.

Warning

You should also give some thought to where you do your module experimentation, development, and testing. We have done our best to make our example modules safe and correct, but the possibility of bugs is always present. Faults in kernel code can bring about the demise of a user process or, occasionally, the entire system. They do not normally create more serious problems, such as disk corruption. Nonetheless, it is advisable to do your kernel experimentation on a system that does not contain data that you cannot afford to lose, and that does not perform essential services. Kernel hackers typically keep a "sacrificial" system around for the purpose of testing new code.

So, if you do not yet have a suitable system with a configured and built kernel source tree on disk, now would be a good time to set that up. We'll wait. Once that task is taken care of, you'll be ready to start playing with kernel modules.

The Hello World Module

Many programming books begin with a "hello world" example as a way of showing the simplest possible program. This book deals in kernel modules rather than programs; so, for the impatient reader, the following code is a complete "hello world" module:

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void)
{
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void)
{
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

This module defines two functions, one to be invoked when the module is loaded into the kernel (*hello_init*) and one for when the module is removed (*hello_exit*). The *module_init* and *m odule_exit* lines use special kernel macros to indicate the role of these two functions. Another special macro (*MODULE_LICENSE*) is used to tell the kernel that this module bears a free license; without such a declaration, the kernel complains when the module is loaded.

The *printk* function is defined in the Linux kernel and made available to modules; it behaves similarly to the standard C library function *printf*. The kernel needs its own printing f unction because it runs by itself, without the he lp of the C library. The module can call *printk* because, after *insmod* has loaded it, the module is linked to the kernel and can access the kernel's public symbols (functions and variables, as detailed in the next section). The string KERN_ALERT is the priority of the message.[1] We've specified a high priority in this module, because a message with the default priority might not show up anywhere useful, depending on the kernel version you are running, the v ersion of the *klogd* daemon, and your configuration. You can ignore this issue for now; we explain it in Chapter 4.

You can test the module with the *insmod* and *rmmod* utilities, as shown below. Note that only the superuser can load and unload a module.

```
  %    make
make[1]: Entering directory `/usr/src/linux-2.6.10'
  CC [M]  /home/ldd3/src/misc-modules/hello.o
  Building modules, stage 2.
  MODPOST
  CC      /home/ldd3/src/misc-modules/hello.mod.o
  LD [M]  /home/ldd3/src/misc-modules/hello.ko
make[1]: Leaving directory `/usr/src/linux-2.6.10'
% su
root# insmod ./hello.ko
Hello, world
root# rmmod hello
Goodbye cruel world
root#
```

Please note once again that, for the above sequence of commands to work, you must have a properly configured and built kernel tree in a place where the makefile is able to find it (*/usr/src/linux-2.6.10* in the example shown). We get into the details of how modules are built in Section 2.4.

According to the mechanism your system uses to deliver the message lines, your output may be different. In particular, the previous screen dump was taken from a text console; if you are running *insmod* and *rmmod* from a terminal emulator running under the window system, you won't see anything on your screen. The message goes to one of the system log files, such as */var/log/messages* (the name of the actual file varies between Linux distributions). The mechanism used to deliver kernel messages is described in Chapter 4.

As you can see, writing a module is not as difficult as you might expect — at least, as long as the module is not required to do an ything wor thwhile. The hard part is understanding your device and how to maximize performance. We go deeper into modularization throughout this chapter and leave device-specific issues for later chapters.

Kernel Modules Versus Applications

Before we go further, it's worth underlining the various differences between a kernel module an d an application.

While most small and medium-sized applications perform a single task from begi nning to e nd, every kernel module just regist ers itself in order to serve future requests, and its initialization function terminates immediately. In other words, the task of the mo dule's ini tialization function is to prepare for later invocation of the module's functions; it's as though the module were saying, "Here I am, and this is what I can do." The module's exit function (*hello_exit* in the example) gets invoked just before the module is unloaded. It should tell the kernel, "I'm not there anymore; don't ask me to do anything else." This kind of approach to programming is similar to event-driven programming, but while not all applications are event-driven, each and every kernel module is. Another major difference between event-driven applications and kernel code is in the exit function: whereas an application that terminates can be lazy in releasing resources or avoids clean up altogether, the exit function of a module must carefully undo everything the *init* function built up, or the pieces remain around until the system is rebooted.

Incidentally, the ability to unload a module is one of the features of modularization that you'll most appreciate, because it helps cut down development time; you can test successive versions of your new driver without going through the lengthy shutdown/reboot cycle each time.

As a programmer, you know that an application can call functions it doesn't define: the linking stage resolves external references using the appropriate library of functions. *printf* is one of those callable functions and is defined in *libc* . A module, on the other hand, is linked only to the kernel, and the only functions it can call are the ones exported by the kernel; there are no libraries to link to. The *printk* function used in *hello.c* earlier, for example, is the version of *printf* defined within the kernel and exported to modules. It behaves similarly to the original function, with a few minor differences, the main one being lack of floating-point support.

Figure 2-1 shows how function calls and function pointers are used in a module to add new functionality to a running kernel.

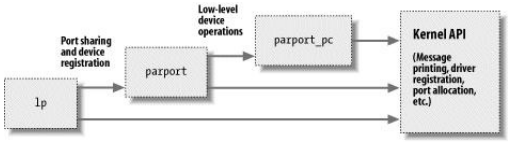

Figure 2-1. Linking a module to the kernel

Because no library is linked to modules, source files should never include the usual header files, <*stdarg.h*> and very special situations being the only exceptions. Only functions that are actually part of the kernel itself may b e used in kernel mod ules. Anyt hing relat ed to the kernel is declared in headers found in the kernel source tree you have set up and configured; most of the relevant headers live in *include/linux* and *include/asm*, but other subdirectories of *include* have been added to host material associated to specific kernel subsystems.

The role of individual kernel headers is introduced throughout the book as each of them is needed.

Another important difference between kernel programming and application programming is in how each environment handles faults: whereas a segmentation fault is harmless during application development and a debugger can always be used to trace the error to the problem in the source code, a kernel fault kills the current process at least, if not the whole system. We see how to trace kernel errors in Chapter 4.

User Space and Kernel Space

A module runs in *kernel space*, whereas applications run in *user spac e*. This concept is at the base of operating systems theory.

The role of the op erating sy stem, in practice, is to provide programs with a consistent view of the computer's hardwa re. In add ition, the operating system must account for independent operation of programs and protection against unauthorized access to resources. This nontrivial task is possible only if the CPU enforces protection of system software from the applications.

Every modern processor is able to enforce this behavior. The chosen approach is to implement different operating modalities (or

levels) in the CPU itself. The levels have different roles, and some operations are disallowed at the lower levels; program code can switch from one level to a nother onl y through a limited number of gates. Unix systems are designed to take advantage of this hardware feature, using two such levels. All current processors have at least two protection levels, and some, like the x86 family, have more levels; when several levels exist, the highest and lowest levels are used. Under Unix, the kernel executes in the highest level (also called *supervisor mode*), where everything is allowed, whereas applications execute in the lowest level (the so-called *user mode*), where the processor regulates direct access to hardware and unauthorized access to memory.

We usually refer to the execution modes as *kernel space* and *user space*. The se terms encompass not only the different privilege levels inherent in the two modes, but also the fact that each mode can have its own memory mapping — it s own address space — as well.

Unix transfers execution from user space to kernel space whenever an application issues a system call or is suspended by a hardware interrupt. Kernel code executing a system call is working in the context of a process — it operates on behalf of the calling process and is able to access data in the process's address space. Code that handles interrupts, on the other hand, is asynchronous with respect to processes and is not related to any particular process.

The role of a module is to extend kernel functionality; modularized code runs in kernel space. Usually a driver performs both the tasks outlined previously: some functions in the module are executed as part of system calls, and some are in charge of interrupt handling.

Concurrency in the Kernel

One way in which kernel programming differs greatly from conventional application programming is the issue of concurrency. Most applications, with the notable exception of multithreading applications, typically run sequentially, from the beginning to the end, without any need to worry about what else might b e happenin g to chang e their environment. Kernel code does not run in such a simple world, and even the simplest kernel modules must be written with the idea that many things can be happening at once.

There are a few sources of concurrency in kernel programming. Naturally, Linux systems run multiple processes, more than one of which can be trying to use your driver at the same time. Most devices are capable of interrupting the processor; interrupt handlers run asynchronously and can be invoked at the same time that your driver is trying to do something else. Several software abstractions (such as kernel timers, introduced in Chapter 7) run asynchronously as well. Moreover, of course, Linux can run on symmetric multiprocessor (SMP) systems, with the result that your driver could be executing concurrently on more than one CPU. Finally, in 2.6, kernel code has been made preemptible; this change causes even uniprocessor systems to have many of the same concurrency issues as multiproces sor system s.

As a result, Linux kernel code, including driver code, must be *reentrant* — it must be capable of running in more than one context at the same time. Data structures must be carefully designed to keep multiple threads of execution separate, and the code must take care to access shared d ata in ways that prevent corruption of the data. Writing code that handles concurrency and a voids race conditions (situations in which an unfortunate order of execution causes undesirable behavior) requires thought and can be tricky. Proper management of concurrency is required to write correct kernel code; for that reason, every sample driver in this book has been written with concurrency in mind. The techniques used are explained as we come to them; Chapter 5 has also been dedicated to this issue and the kernel primitives available for concurrency management.

A common mistake made by driver programmers is to assume that concurrency is not a problem as long as a particular segment of code does not go to sleep (or "block"). Even in previous kernels (which were not preemptive), this assumption was not valid on multiprocessor systems. In 2.6, kernel code can (almos t) never assume that it can hold the processor over a given stretch of code. If you do not write your code with concurrency in mind, it will be subject to catastrophic failures that can be exceedingly difficult to debug.

The Current Process

Although kernel modules don't execute sequentially as applications do, most actions performed by the kernel are done on beha lf of a specific process. Kernel code can refer to the current process by accessing the global item `curre nt` , defined in *< asm/curren t.h>* , which yields a pointer to `struct task_struct`, defined by *<linux/sched.h>*. The `current` pointer refers to the process that is currently executing. During the execution of a system call, such as *open* or *read*, the current process is the one that invoked the call. Kernel code can use process-specific information by using `current`, if it needs to do so. An example of this technique is presented in Chapter 6.

Actually, `current` is not truly a global variable. The need to support SMP systems forced the kernel developers to develop a mechanism that finds the current process on the relevant CPU. This mechanism must also be fast, since references to `current` happen frequently. The result is an architecture-dependent mechanism that, usually, hides a pointer to the `task_struct` structure on the kernel stack. The details of the implementation remain hidden to other kernel subsystems though, and a device driver can just include *<linux/sched.h>* and refer to the `current` process. For example, the following statement prints the process ID and the command name of the current process by accessing certain fields in `struct task_struct`:

```
printk(KERN_INFO "The process is \"%s\" (pid %i)\n",
       current->comm, current->pid);
```

The command name stored in `current->comm` is the base name of the program file (trimmed to 15 characters if need be) that is being executed by the current process.

A Few Other Details

Kernel programming differs from user-space programming in many ways. We'll point things out as we get to them over the course of the book, but there are a few fundamental issues which, while not warranting a section of their own, are worth a mention. So, as you dig into the kernel, the following issues should be kept in mind.

Applications are laid out in virtual memory with a very large stack area. The stack, of course, is used to hold the function call history and all automatic variables created by currently active functions. The kernel, instead, has a very small stack; it can be as small as a single, 4096-byte page. Your functions must share that stack with the entire kernel-space call chain. Thus, it is never a good idea to declare large automatic variables; if you need larger structures, you should allocate them dynamically at call time.

Often, as you look at the kernel API, you will encounter function names starting with a double underscore (_ _). Functions so marked are generally a low-level component of the interface and should be used with caution. Essentially, the double underscore says to the programmer: “If you call this function, be sure you know what you are do ing.”

Kernel code cannot do floating point arithmetic. Enabling floating point would require that the kernel save and restore the floating point processor’s state on each entry to, and exit from, kernel space — at least, on some architectures. Given that there really is no need for floating point in kernel code, the extra overhead is not worthwhile.

Compiling and Loading

The “hello world” example at the begin ning of this chapter included a brief demonstration of building a module and loading it into the system. There is, of course, a lot more to that whole process than we have seen so far. This section provides more detail on how a module author turns source code into an executing subsystem within the kernel.

Compiling Modules

As the first step, we need to look a bit at how modules must be built. The build process for modules differs significantly from that used for user-space applications; the kernel is a large, s tandalone program with detailed and explicit requirements on how its pieces are put together. The build process also differs from how things were done with previous v ersions of the kernel; the new build system is simpler to use and produces more correct results, but it looks very different from what came before. The kernel build system is a complex beast, and we just look at a tiny piece of it. The files found in the *Documentation/kbuild* directory in the kernel source are required reading for anybody wanting to understand all that is really going on beneath the surface.

There are some prerequisites that you must get out of the way before you can build kernel modules. The first is to ensure that you have sufficiently current versions of the compiler, module utilities, and other necessary tools. The file *Documentation/Changes* in the kernel documentation directory always lists the required tool versions; you should consult it before going any further. Trying to build a kernel (and its modules) with the wrong tool versions can lead to no end of subtle, difficult problems. Note that, occasionally, a version of the compiler that is too new can be just as problematic as one that is too old; the kernel source makes a great many assumptions about the compiler, and new releases can sometimes break things for a while.

If you still do not have a kernel tree handy, or have not yet configured and built that kernel, now is the time to go do it. You cannot build loadable modules for a 2.6 kernel without this tree on your filesystem. It is also helpful (though not required) to be actually running the kernel that you are building for.

Once you have everything set up, creating a makefile for your module is straightforward. In fact, for the “hello world” example shown earlier in this chapter, a single line will suffice:

```
obj-m := hello.o
```

Readers who are familiar with *make*, but not with the 2.6 kernel build system, are likely to be wondering how this makefile works. The above line is not how a traditional makefile looks, after all. The answer, of course, is that the kernel build system handles the rest. The assignment above (which takes advantage of the extended syntax provided by GNU *make*) states that there is one module to be built from the object file *hello.o*. The resulting module is named *hello.ko* after being built from the object file.

If, instead, you have a module called *module.ko* that is generated from two source files (called, say, *file1.c* and *file2.c*), the correct incantation would be:

```
obj-m := module.o
module-objs := file1.o file2.o
```

For a makefile like those shown above to work, it must be invoked within the context of the larger kernel build system. If your kernel source tree is located in, say, your *~/kernel-2.6* directory, the *make* command required to build your module (typed in the directory containing the module source and makefile) would be:

```
make -C ~/kernel-2.6 M=`pwd` modules
```

This command starts by changing its directory to the one provided with the `-C` option (that is, your kernel source directory). There it finds the kernel’s top-level makefile. The `M=` option causes that makefile to move back into your module source directory before trying to build the `modules` target. This target, in turn, refers to the list of modules found in the `obj-m` variable, which we’ve set to *module.o* in our examples.

Typing the previous *make* command can get tiresome after a while, so the kernel developers have developed a sort of makefile idiom, which makes life easier for those building modules outside of the kernel tree. The trick is to write your makefile as follows:

```
# If KERNELRELEASE is defined, we've been invoked from the
# kernel build system and can use its language.
ifneq ($(KERNELRELEASE),)
    obj-m := hello.o

# Otherw ise we were called directly from the command
# line; invoke the kernel build system.
else

    KERNELDIR ?= /lib/modules/$(shell uname -r)/build
    PWD  := $(shell pwd)

default:
    $(MAKE) -C $(KERNELDIR) M=$(PWD) modules

endif
```

Once again, we are seeing the extended GNU *make* syntax in action. This makefile is read twice on a typical build. When the makefile is invoked from the command line, it notices that the `KERNELRELEASE` variable has not been set. It locates the kernel source directory by taking advantage of the fact that the symbolic link *build* in the installed modules directory points back at the kernel build tree. If you are not actually running the kernel that you are building for, you can supply a `KERNELDIR=` option on the command

line, set the KERNELDIR environment variable, or rewrite the line that sets KERNELDIR in the makefile. Once the kernel source tree has been found, the makefile invokes the default: target, which runs a second *make* command (parameterized in the makefile as $(MAKE)) to invoke the kernel build system as described previously. On the second reading, the makefile sets obj-m, and the kernel makefiles take care of actually building the module.

This mechanism for building modules may strike you as a bit unwieldy and obscure. Once you get used to it, however, you will likely appreciate the capabilities that have been programmed into the kernel build system. Do note that the above is not a complete makefile; a real makefile includes the usual sort of targets for cleaning up unneeded files, installing modules, etc. See the makefiles in the example source directory for a complete example.

Loading and Unloading Modules

After the module is built, the next step is loading it into the kernel. As we've already pointed out, *insmod* does the job for you. The program loads the module code and data into the kernel, which, in turn, performs a function similar to that of *ld*, in that it links any unresolved symbols in the module to the symbol table of the kernel. Unlike the linker, however, the kernel doesn't modify the module's disk file, but rather an in-memory copy. *insmod* accepts a number of command-line options (for details, see the manpage), and it can assign values to parameters in your module before linking it to the current kernel. Thus, if a module is correctly designed, it can be configured at load time; load-time configuration gives the user more flexibility than compile-time configuration, which is still used sometimes. Load-time configuration is explained in Section 2.8 later in this chapter.

Interested readers may want to look at how the kernel supports *insmod*: it relies on a system call defined in *kernel/module.c*. The function *sys_init_module* allocates kernel memory to hold a module (this memory is allocated with *vmalloc*; see the Section 8.4 in Chapter 2); it then copies the module text into that memory region, resolves kernel references in the module via the kernel symbol table, and calls the module's initialization function to get everything going.

If you actually look in the kernel source, you'll find that the names of the system calls are prefixed with sys_. This is true for all system calls and no other functions; it's useful to keep this in mind when grepping for the system calls in the sources.

The *modprobe* utility is worth a quick mention. *modprobe*, like *insmod*, loads a module into the kernel. It differs in that it will look at the module to be loaded to see whether it references any symbols that are not currently defined in the kernel. If any such references are found, *modprobe* looks for other modules in the current module search path that define the relevant symbols. When *modprobe* finds those modules (which are needed by the module being loaded), it loads them into the kernel as well. If you use *insmod* in this situation instead, the command fails with an "unresolved symbols" message left in the system logfile.

As mentioned before, modules may be removed from the kernel with the *rmmod* utility. Note that module removal fails if the kernel believes that the module is still in use (e.g., a program still has an open file for a device exported by the modules), or if the kernel has been configured to disallow module removal. It is possible to configure the kernel to allow "forced" removal of modules, even when they appear to be busy. If you reach a point where you are considering using this option, however, things are likely to have gone wrong badly enough that a reboot may well be the better course of action.

The *lsmod* program produces a list of the modules currently loaded in the kernel. Some other information, such as any other modules making use of a specific module, is also provided. *lsmod* works by reading the */proc/modules* virtual file. Information on currently loaded modules can also be found in the sysfs virtual filesystem under */sys/module*.

Version Dependency

Bear in mind that your module's code has to be recompiled for each version of the kernel that it is linked to — at least, in the absence of modversions, not covered here as they are more for distribution makers than developers. Modules are strongly tied to the data structures and function prototypes defined in a particular kernel version; the interface seen by a module can change significantly from one kernel version to the next. This is especially true of development kernels, of course.

The kernel does not just assume that a given module has been built against the proper kernel version. One of the steps in the build process is to link your module against a file (called *vermagic.o*) from the current kernel tree; this object contains a fair amount of information about the kernel the module was built for, including the target kernel version, compiler version, and the settings of a number of important configuration variables. When an attempt is made to load a module, this information can be tested for compatibility with the running kernel. If things don't match, the module is not loaded; instead, you see something like:

```
# insmod hello.ko
Error inserting './hello.ko': -1 Invalid module format
```

A look in the system log file (*/var/log/messages* or whatever your system is configured to use) will reveal the specific problem that caused the module to fail to load.

If you need to compile a module for a specific kernel version, you will need to use the build system and source tree for that particular version. A simple change to the KERNELDIR variable in the example makefile shown previously does the trick.

Kernel interfaces often change between releases. If you are writing a module that is intended to work with multiple versions of the kernel (especially if it must work across major releases), you likely have to make use of macros and #ifdef constructs to make your code build properly. This edition of this book only concerns itself with one major version of the kernel, so you do not often see version tests in our example code. But the need for them does occasionally arise. In such cases, you want to make use of the definitions found in *linux/version.h*. This header file, automatically included by *linux/module.h*, defines the following macros:

UTS_RELEASE

> This macro expands to a string describing the version of this kernel tree. For example, "2.6.10".

LINUX_VERSION_CODE

> This macro expands to the binary representation of the kernel version, one byte for each part of the version release number. For example, the code for 2.6.10 is 132618 (i.e., 0x02060a).[2] With this information, you can (almost) easily determine what version of the kernel you are dealing with.

KERNEL_VERSION(major,minor,release)

This is the macro used to build an integer version code from the individual numbers that build up a version number. For example, `KERNEL_VERSION(2,6,10)` expands to 132618. This macro is very useful when you need to compare the current version and a known che ckpoint.

Most dependencies based on the kernel version can be worked around with preprocessor conditionals by exploiting `KERNEL_VERSION` and `LINUX_VERSION_CODE`. Version dependency should, however, not clutter driver code with hairy `#ifdef` conditionals; the best way to deal with incompatibilities is by confining them to a specific header file. As a general rule, code which is explicitly version (or platform) dependent should be hidden behind a low-level macro or function. High-level code can then just call those functions without concern for the low-level details. Code written in this way tends to be easier to read and more robust.

Platform Dependency

Each computer platform has its peculiarities, and kernel designers are free to exploit all the peculiarities to achieve better performance in the target object file.

Unlike application developers, who must link their code with precompiled libraries and stick to convention s on parameter passing, kernel developer s can dedi cate some processor registers to specific roles, and they have done so. Moreover, kernel code can be optimized for a specific processor in a CPU family to get the best from the target platform: unlike applications that are often distributed in binary format, a custom compilation of the kernel can be optimized for a specific computer set.

For example, the IA32 (x86) architecture has been subdivided into several different processor types. The old 80386 processor is still supported (for now), even though its instruction set is, by modern standards, quite limited. The more modern processors in this architecture have introduced a number of new capabilities, including faster instructions for entering the kernel, interprocessor locking, copying data, etc. Newer processors can also, when operated in the correct mode, employ 36-bit (or larger) physical addresses, allowing them to address more than 4 GB of physical memory. Other processor families have seen similar improvements. The kernel, depending on various configuration options, can be built to make use of these additional features.

Clearly, if a module is to work with a given kernel, it must be built with the same understanding of the target processor as that kernel was. Once again, the *vermagic.o* object comes in to play. When a module is loaded, the kernel checks the processor-specific configuration options for the module and makes sure they match the running kernel. If the module was compiled with different options, it is not loaded.

If you are planning to write a driver for general distribution, you may well be wondering just how you can possibly support all these different variations. The best answer, of course, is to release your driver under a GPL-compatible license and contribute it to the mainline kernel. Failing that, distributing your driver in source form and a set of scripts to compile it on the user's system may be th e best ans wer. Some vendors have released tools to make this task easier. If you must distribute your driver in binary form, you need to look at the different kernels provided by your target distributions, and provide a version of the module for each. Be sure to take into account any errata kernels that may have been released since the distribution was produced. Then, there are licensing issues to be considered, as we discussed in Section 1.6. As a general rule, distributing things in source form is an easier way to make your way in the world.

The Kernel Symbol Table

We've seen how *insmod* resolves undefined symbols against the table of public kernel symbols. The table contains the addresses of global kernel items — fun ctions and variables — that are needed to implement modularized drivers. When a module is loaded, any symbol exported by the module becomes part of the kernel symbol table. In the usual case, a module imp lements it s own func tionality without th e need to export any symbols at all. You need to export symbols, however, whenever other modules may benefit from using them.

New modules can use symbols exported by your module, and you can stack new modules on top of other modules. Module stacking is implemented in the mainstream kernel sources as well: the *msdos* filesystem relies on symbols exported by the *fat* module, and e ach input USB device module st acks on th e *usbco re* and *input* modules.

Module stacking is useful in complex projects. If a new abstraction is implemented in the form of a device driver, it might offer a plug for hardware-specific implementations. For example, the video-for-linux set of drivers is split into a generic module that exports symbols used by lower-level device drivers for specific hardware. According to your setup, you load the generic video modu le and the specific module for your installed hardware. Support for parallel ports and the wide variety of attachable devices is handled in the same way, as is the USB kernel subsystem. Stacking in the parallel port subsystem is shown in Figure 2-2; the arrows show the communications between the modules and with the kernel programming interface.

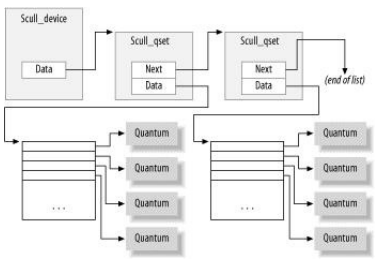

Figure 2-2. Stacking of parallel port driver modules

When using stacked modules, it is helpful to be aware of the *modprobe* utility. As we described earlier, *modprobe* functions in much the same way as *insmod*, but it also loads any other modules that are required by the module you want to load. Thus, one *modprobe* command can sometimes replace several invocations of *insmod* (altho ugh you'll still need *insmod* when loading your own modules from the current directory, because *modprobe* looks only in the standard installed module directories).

Using stacking to split modules into multiple layers can help reduce development time by simplifying each layer. This is similar

to the separation between mechanism and policy that we discussed in Chapter 1.

The Linux kernel header files provide a convenient way to manage the visibility of your symbols, thus reducing namespace pollution (filling the namespace with names that may conflict with those defined elsewhere in the kernel) and promoting proper information hiding. If your module needs to export symbols for ot her modules to use, the following macros should be used.

```
EXPORT_SYMBOL(name);
EXPORT_SYMBOL_GPL(name);
```

Either of the above macros makes the given symbol available outside the module. The `_GPL` version makes the symbol available to GPL-licensed modules only. Symbols must be exported in the global part of the module's file, outside of any function, because the macros expand to the declaration of a special-purpose variable that is expected to be accessible globally. This variable is stored in a special part of the module executible (an "ELF section") that is used by the kernel at load time to find the variables exported by the module. (Interested readers can look at *<linux/module.h>* for the details, even though the details are not needed to make things work.)

Preliminaries

We are getting closer to looking at some actua l module c ode. But f irst, we n eed to loo k at some other things that need to appear in your module source files. The kernel is a unique environment, and it imposes its own re quirements on code that would interface with it.

Most kernel code ends up including a fairly large numb er of head er files t o get defi nitions of functions, data types, and variables. We'll examine these files as we come to them, but there are a few that are specific to modules, and must appear in every loadable module. Thus, just about all module code has the following:

```
#include <linux/module.h>
#include <linux/init.h>
```

module.h contains a great many definitions of symbols and functions needed by loadable modules. You need *init.h* to specify your initialization and cleanup functions, as we saw in the "hello world" example above, and which we revisit in the next section. Most modules also include *moduleparam.h* to enable the passing of parameters to the module at load time; we will get to that shortly.

It is not strictly necessary, but your module really should specify which license applies to its code. Doing so is just a matter of including a `MODULE_LICENSE` line:

```
MODULE_LICENSE("GPL");
```

The specific licenses recognized by the kernel are "GPL" (for any version of the GNU General Public License), "GPL v2" (for GPL version two only), "GPL and additional rights," "Dual BSD/GPL," "Dual MPL/GPL," and "Proprietary." Unless your module is explicitly marked as being under a free license recognized by the kernel, it is assumed to be proprietary, and the kernel is "tainted" when the module is loaded. As we mentioned in Section 1.6, kernel developers tend to be unenthusiastic about helping users who experience problems after loading proprietary modules.

Other descriptive definitions that can be contained within a module include `MODULE_AUTHOR` (stating who wrote the module), `MODULE_DESCRIPTION` (a human-readable statement of what the module does), `MODULE_VERSION` (for a code revision number; see the comments in *<linux/module.h>* for the conventions to use in creating version strings), `MODULE_ALIAS` (another name by which this module can be known), and `MODULE_DEVICE_TABLE` (to tell user space about which devices the module supports).

The various `MODULE_` declarations can appear anywhere within your source file outside of a function. A relatively recent convention in kernel code, however, is to put these declarations at the end of the file.

Initialization and Shutdown

As already mentioned, the module initialization function registers any facility offered by the module. By *facility*, we mean a new functionality, be it a whole driver or a new software abstraction, that can be accessed by an application. The actual definition of the initialization function always looks like:

```
    static int __init initialization_function(void)
{
    /* Initialization code here */
}
module_init(initialization_function);
```

Initialization functions should be declared `static`, since they are not meant to be visible outside the specific file; there is no hard rule about this, though, as no function is exported to the rest of the kernel unless explicitly requested. The `__init` token in the definition may look a little strange; it is a hint to the kernel that the given function is used only at initialization time. The module loader drops the initialization function after the module is loaded, making its memory available for other uses. There is a similar tag (`__initdata`) for data used only during initialization. Use of `__init` and `__initdata` is optional, but it is worth the trouble. Just be sure not to use them for any function (or data structure) you will be using after initialization completes. You may also encounter `__devinit` and `__devinitdata` in the kernel source; these translate to `__init` and `__initdata` only if the kernel has not been configured for hotpluggable devices. We will look at hotplug support in Chapter 14.

The use of *module_init* is mandatory. This macro adds a special section to the module's object code stating where the module's initialization function is to be found. Without this definition, your initialization function is never called.

Modules can register many different types of facilities, including different kinds of devices, filesystems, cryptographic transforms, and more. For each facility, there is a specific kernel function that accomplishes this registration. The arguments passed to the kernel registration functions are usually pointers to data structures describing the new facility and the name of the facility being registered. The data structure usually contains pointers to module functions, which is how functions in the module body get called.

The items that can be registered go beyond the list of device types mentioned in Chapter 1. They include, among others, serial ports, miscellaneous devices, sysfs entries, */proc* files, executable domains, and line disciplines. Many of those registrable items support functions that aren't directly related to hardware but remain in the "software abstractions" field. Those items can be registered, because they are integrated into the driver's functionality anyway (like */proc* files and line disciplines for example).

There are other facilities that can be registered as add-ons for certain drivers, but their use is so specific that it's not worth

talking about them; they use the stacking technique, as described in Section 2.5. If you want to probe further, you can grep for `EXPORT_SYMBOL` in the kernel sources, and find the entry points off ered by di fferent drivers. Most registration functions are prefixed with `register_`, so another possible way to find them is to grep for `register_` in the kernel source.

The Cleanup Function

Every nontrivial module also requires a cleanup function, which unregisters interfaces and returns all resources to the system before the module is removed. This function is defined as:

```
 static void _ _exit cleanup_function(void)
{
    /* Cleanup code here */
}

module_exit(cleanup_function);
```

The cleanup function has no value to return, so it is declared `void`. The `_ _exit` modifier marks the code as being for module unload only (by causing the compiler to place it in a special ELF section). If your module is built directly into the kernel, or if your kernel is configured to disallow the unloading of modules, functions marked `_ _exit` are simply discarded. For this reason, a function marked `_ _exit` can be called *only* at module unload or system shutdown time; any other use is an error. Once again, the *module_exit* declaration is necessary to enable to kernel to find your cleanup function.

If your module does not define a cleanup function, the kernel does not allow it to be unloaded.

Error Handling During Initialization

One thing you must always bear in mind when registering facilities with the kernel is that the registration could fail. Even the simplest action often requires memory allocation, and the required memory may not be available. So module code must always check return values, and be sure that the requested operations have actually succeeded.

If any errors occur when you register utilities, the first orde r of business is to decide whether the module can continue initializing itself anyway. Often, the module can continue to operate after a registration failure, with degraded functionality if necessary. Whenever possible, your module should press forward and provide what capabilities it can after things fail.

If it turns out that your module simply cannot load after a particular type of failure, you must undo any registration activities performed before the failure. Linux doesn't keep a per-module registry of facilities that have been registered, so the module must back out of everything itself if initialization fails at some point. If you ever fail to unregister what you obtained, the kernel is left in an unstable state; it contains internal pointers to code that no longer exists. In such situations, the only recourse, usually, is to reboot the system. You really do want to take care to do the right thing when an initialization error occurs.

Error recovery is sometimes best handled with the `goto` statement. We normally hate to use `goto`, but in our opinion, this is one situation where it is useful. Careful use of `goto` in error situations can eliminate a gr eat deal of complicated, highly-inden ted, "stru ctured" lo gic. Thus, in the kernel, `goto` is often used as shown here to deal with errors.

The following sample code (using fictitious registration and unregistration functions) behaves correctly if initialization fails at any point:

```
int _ _init my_init_function(void)
{
    int err;

    /* registration takes a pointer and a name */
    err = register_this(ptr1, "skull");
    if (err) goto fail_this;
    err = register_that(ptr2, "skull");
    if (err) goto fail_that;
    err = register_those(ptr3, "skull");
    if (err) goto fail_those;

    return 0; /* success */

  fail_those: unregister_that(ptr2, "skull");
  fail_that: unregister_this(ptr1, "skull");
  fail_this: return err; /* propagate the error */
 }
```

This code attempts to register three (fictitious) facilities. The `goto` statement is used in case of failure to cause the unregistration of only the facilities that had been successfully registered before things went bad.

Another option, requiring no hairy `goto` statements, is keeping track of what has been successfully registered and calling your module's cleanup function in case of any error. The cleanup function unrolls only the steps that have been successfully accomplished. This alternative, however, requires more code and more CPU time, so in fast paths y ou still r esort to `goto` as the best error-recovery tool.

The return value of *my_init_function*, `err`, is an error code. In the Linux kernel, error codes are negative numbers belonging to the set defined in *<linux/errno.h>*. If you want to generate your own error codes instead of returning what you get from other functions, you should include *<linux/errno.h>* in order to use symbolic value s such as `-ENODEV`, `-ENOMEM`, and so on. It is always good practice to return appropriate error codes, because user programs can turn them to meaningful strings using *perror* or similar means.

Obviously, the module cleanup function must undo any registration performed by the initialization function, and it is customary (but not usually mandatory) to unregister facilities in the reverse order used to register them:

```
void _ _exit my_cleanup_function(void)
{
    unregister_those(ptr3, "skull");
    unregister_that(ptr2, "skull");
    unregister_this(ptr1, "skull");
    return;
}
```

If your initialization and cleanup are more complex than dealing with a few items, the `goto` approach may become difficult to manage, because all the cleanup code must be repeated within the initialization function, with several labels intermixed. Sometimes, therefore, a different layout of the code proves more successful.

What you'd do to minimize code duplication and keep everything streamlined is to call the cleanup function from within the initialization whenever an error occurs. The cleanup function then must check the status of each item before undoing its registration. In its simplest form, the code looks like the following:

```
struct something *item1;
struct somethingelse *item2;
int stuff_ok;

void my_cleanup(void)
{
    if (item1)
        release_thing(item1);
    if (item2)
        release_thing2(item2);
    if (stuff_ok)
        unregister_stuff(  );
    return;
 }

int _ _init my_init(void)
{
    int err = -ENOMEM;

    item1 = allocate_thing(arguments);
    item2 = allocate_thing2(arguments2);
    if (!item2 || !item2)
        goto fail;
    err = register_stuff(item1, item2);
    if (!err)
        stuff_ok = 1;
    else
        goto fail;
    return 0; /* success */

  fail:
    my_cleanup(  );
    return err;
}
```

As shown in this code, you may or may not need external flags to mark success of the initialization step, depending on the semantics of the registration/allocation function you call. Whether or not flags are needed, this kind of initialization scales well to a large number of items and is often better than the technique shown earlier. Note, however, that the cleanup function cannot be marked `_ _exit` when it is called by nonexit code, as in the previous example.

Module-Loading Races

Thus far, our discussion has skated over an important aspect of module loading: race conditions. If you are not careful in how you write your initialization function, you can create situations that can compromise the stability of the system as a whole. We will discuss race condit ions later in this book; for now, a couple of quick points will have to suffice.

The first is that you should al ways remem ber that so me other part of the kernel can make use of any facility you register immediately after that registration has completed. It is entirely possible, in other words, that the kernel will make calls into your module while your initialization function is still running. So your code must be prepared to be called as soon as it completes its first registration. Do not register any facility until all of your internal initialization neede d to support that facility has been completed.

You must also consider what happens if your initialization function decides to fail, but some part of the kernel is already making use of a facility your module has registered. If this situation is possible for your module, you should seriously consider not failing the initialization at all. After all, the module has clearly succeeded in exporting something useful. If initialization must fail, it must carefully step around any possible operations going on elsewhere in the kernel until those operations have completed.

Module Parameters

Several parameters that a driver needs to know can change from system to system. These c an vary fr om the dev ice number to use (as we'll see in the next chapter) to numerous aspects of how the driver should operate. For example, drivers for SCSI adapters often have options controlling the use of tagged command queuing, and the Integrated Device Electronics (IDE) drivers allow user control of DMA operations. If your driver controls older h ardware, i t may also need to b e told exp licitly where to find that hardware's I/O ports or I/O memory addresses. The kernel supports these needs by making it possible for a driver to designate parameters that may be changed when the driver's module is loaded.

These parameter values can be assigned at load time by *insmod* or *modprobe* ; the latter can also read parameter assignment from its configuration file (*/etc/modprobe.conf*). The commands accept the specification of several types of values on the command line. As a way of demonstrating this capability, imagine a much-needed enhancement to the "hello world" module (ca lled *hellop*) shown at the beginning of this chapter. We add two parameters: an integer value called `howmany` and a character string called `whom`. Our vastly more functional module then, at load time, greets `whom` not just once, but `howmany` times. Such a module could then be loaded with a command line such as:

```
insmod hellop howmany=10 whom="Mom"
```

Upon being loaded that way, *hellop* would say "Hello, Mom" 10 times.

However, before *insmod* can change module parameters, the module must make them available. Parameters are declared with the `module_param` macro, which is defined in *moduleparam.h*. `module_param` takes three parameters: the name of the variable, its type, and a permissions mask to be used for a n accompanying sysfs entry. The macro should be placed outside of any function and is typically found near the head of the source file. So *hellop* would declare its parameters and make them available to *insmod* as follows:

```
static char *whom = "world";
static int howmany = 1;
module_param(howmany, int, S_IRUGO);
module_param(whom, charp, S_IRUGO);
```

Numerous types are supported for module parameters:

```
bool
invbool
```

A boolean (true or false) value (the associated variable should be of type `int`). The `invbool` type inverts the value, so that true values become false and vice versa.

`charp`

A char pointer value. Memory is allocated for user-provided strings, and the pointer is set accordingly.

`int`
`long`
`short`
`uint`
`ulong`
`ushort`

Basic integer values of various lengths. The versions starting with `u` are for unsigned values.

Array parameters, where the values are supplied as a comma-separated list, are also supported by the module loader. To declare an array parameter, use:

```
module_param_array(name,type,nump,perm);
```

Where `name` is the name of your array (and of the parameter), `type` is the type of the array elements, `nump` is an integer variable, and `perm` is the usual permissions value. If the array parameter is set at load time, `nump` is set to the number of values supplied. The module loader refuses to accept more values than will fit in the array.

If you really need a type that does not appear in the list above, there are hooks in the module code that allow you to define them; see *moduleparam.h* for details on how to do that. All module parameters should be given a default value; *insmod* changes the value only if explicitly told to by the user. The module can check for explicit parameters by testing parameters against their default values.

The final *module_param* field is a permission value; you should use the definitions found in *<linux/stat.h>*. This value controls who can access the representation of the module parameter in sysfs. If `perm` is set to `0`, there is no sysfs entry at all; otherwise, it appears under */sys/module* [3] with the given set of permissions. Use `S_IRUGO` for a parameter that can be read by the world but cannot be changed; `S_IRUGO|S_IWUSR` allows root to change the parameter. Note that if a parameter is changed by sysfs, the value of that parameter as see n by your module changes, but your module is not notified in any other way. You should probably not make module parameters writable, unless you are prepared to detect the change and react accordingly.

Doing It in User Space

A Unix programmer who's addressing kernel issues for the first time might be nervous about writing a module. Writing a u ser program that reads and writes directly to the device ports may be easier.

Indeed, there are some arguments in favor of user-space programming, and sometimes writing a so-called user-space device driver is a wise altern ative to k ernel hack ing. In th is section, we discuss some of the reasons why you might write a driver in user space. This book is about kernel-space drivers, however, so we do n ot go beyond this introductory discussion.

The advantages of user-space drivers are:

- The full C library can be linked in. The driver can perform many exotic tasks without resorting to external programs (the utility programs implementing usage policies that are usually distributed along with the driver itself).
- The programmer can run a conventional debugger on the driver code without having to go through contortions to debug a running kernel.
- If a user-space driver hangs, you can simply kill it. Problems with the driver are unlikely to hang the entire system, unless the hardware being controlled is *really* misbehaving.
- User memory is swappable, unlike kernel memory. An infrequently used device with a huge driver won't occupy RAM that other programs could be using, except when it is actually in use.
- A well-designed driver program can still, like kernel-space drivers, allow concurrent access to a device.
- If you must write a closed-source driver, the user-space option makes it easier for you to avoid ambiguous licensing situations and problems with changing kernel interfaces.

For example, USB drivers can be written for user space; see the (still young) libusb project at libusb.sourceforge.net and "gadgetfs" in the kernel source. Another example is the X server: it knows exactly what the hardware can do and what it can't, and it offers the graphic resources to all X clients. Note, however, that there is a slow but steady drift toward frame-buffer-based graphics environments, where the X server acts only as a server based on a real kernel-space device driver for actual graphic manipulation.

Usually, the writer of a user-space driver implements a server process, taking over from the kernel the task of being the single agent in charge of hardware control. Client applications can then connect to the server to perform actual communication with the device; therefore, a smart driver process can allow concurrent access to the device. This is exactly how the X server works.

But the user-space approach to device driving has a number of drawbacks. The most important are:

- Interrupts are not available in user space. There are workarounds for this limitation on some platforms, such as the *vm86* system call on the IA32 architecture.
- Direct access to memory is possible only by *mmap*ping */dev/mem*, and only a privileged user can do that.
- Access to I/O ports is available only after calling *ioperm* or *iopl*. Moreover, not all platforms support these system calls, and access to */dev/port* can be too slow to be effective. Both the system calls and the device file are reserved to a privileged user.
- Response time is slower, because a context switch is required to transfer information or actions between the client and the hardware.
- Worse yet, if the driver has been swapped to disk, response time is unacceptably long. Using the *mlock* system call might help, but usually you'll need to lock many memory pages, because a user-space program depends on a lot of library code. *mlock*, too, is limited to privileged users.
- The most important devices can't be handled in user space, including, but not limited to, network interfaces and block devices.

As you see, user-space drivers can't do that much after all. Interesting applications nonetheless exist: for example, support for SCSI scanner devices (implemented by the *SANE* package) and CD writers (implemented by *cdrecord* and other tools). In both cases, user-level device drivers rely on the "SCSI generic" kernel driver, which exports low-level SCSI functionality to user-space programs so they can drive their own hardware.

One case in which working in user space might make sense is when you are beginning to deal with new and unusual hardware. This way you can learn to manage your hardware without the risk of hanging the whole system. Once you've done that, encapsulating the software in a kernel module should be a painless operation.

Quick Reference

This section summarizes the kernel functions, variables, macros, and */proc* files that we've touched on in this chapter. It is meant to act as a reference. Each item is listed after the r elevant he ader file, if any. A similar section app ears at the end of almost every chapter from here on, summarizing the new symbols introduced in the chapter. Entri es in this section generally appear in the same order in which they were introduced in the chapter:

insmod
modprobe
rmmod

User-space utilities that load modules into the running kernels and remove them.

```
#include <linux/init.h>
module_init(init_function);
module_exit(cleanup_function);
```

Macros that designate a module's initialization and cleanup functions.

```
_ _init
_ _initdata
_ _exit
_ _exitdata
```

Markers for functions (`_ _init` and `_ _exit`) and data (`_ _initdata` and `_ _exitdata`) that are only used at module initialization or cleanup time. Items marked for initialization may be discarded once initialization completes; the exit items may be discarded if module unloading has not been configured into the kernel. These markers work by causing the relevant objects to be placed in a special ELF section in the executable file.

```
#include <linux/sched.h>
```

One of the most important header files. This file contains definitions of much of the kernel API used by the driver, including functions for sleeping and numerous variable declarations.

```
struct task_struct *current;
```

The current process.

```
 current->pid
current->comm
```

The process ID and command name for the current process.

```
obj-m
```

A makefile symbol used by the kernel build system to determine which modules should be built in the current directory.

/sys/module
/proc/modules

/sys/module is a sysfs directory hierarchy containing information on currently-loaded modules. */proc/modules* is the older, single-file version of that information. Entries contain the module name, the amount of memory each module occupies, and the usage count. Extra strings are appended to each line to specify flags that are currently active for the module.

vermagic.o

An object file from the kernel source directory that describes the environment a module was built for.

```
#include <linux/module.h>
```

Required header. It must be included by a module source.

```
#include <linux/version.h>
```

A header file containing information on the version of the kernel being built.

```
LINUX_VERSION_CODE
```

Integer macro, useful to `#ifdef` version dependencies.

```
EXPORT_SYMBOL (symbol) ;
EXPORT_SYMBOL_GPL (symbol);
```

Macro used to export a symbol to the kernel. The second form lim its use of the exported symbol to GPL-licensed modules.

```
MODULE_AUTHOR(author);
MODULE_DESCRIPTION(description);
MODULE_VERSION(version_string);
 MODULE_DEVICE_TABLE(table_info);
MODULE_ALIAS(alternate_name);
```

Place documentation on the module in the object file.

```
MODULE_LICENSE(license);
```

Declare the license governing this module.

```
#include <linux/moduleparam.h>
    module _param(variable, type, perm);
```

Macro that creates a module parameter that can be adjusted by the user when the module is loaded (or at boot time for built-in code). The type can be one of `bool`, `charp`, `int`, `invbool`, `long` , `short`, `ushort`, `uint`, `ulong`, or `intarray`.

```
#include <linux/kernel.h>
int printk(const char * fmt, ...);
```

The analogue of *printf* for kernel code.

---

[1] The priority is just a string, such as `<1>`, which is prepended to the *printk* format string. Note the lack of a comma after `KERN_ALERT`; adding a comma there is a common and annoying typo (which, fortunately, is caught by the compiler).

[2] This allows up to 256 development versions between stable versions.

[3] As of this writing, there is talk of moving parameters elsewhere within sysfs, however.

Chapter 5. Concurrency and Race Conditions

Thus far, we have paid little attention to the problem of concurrency — i.e., what happens when the system tries to do more than one thing at once. The management of concurrency is, however, one of the core problems in operating systems programming. Concurrency-related bugs are some of the easiest to create and some of the hardest to find. Even expert Linux kernel programmers end up creating concurrency-related bugs on occasion.

In early Linux kernels, there were relatively few sources of concurrency. Symmetric multiprocessing (SMP) systems were not supported by the kernel, and the only cause of concurrent execution was the servicing of hardware interrupts. That approach offers simplicity, but it no longer works in a world that prizes performance on systems with mo re and more processors, and that insists that the system respond to events quickly. In response to the demands of modern hardware and applications, the Linux kernel has evolved to a point where many more things are going on simultaneously. This evolution has resulted in far greater performance and scalability. It has also, however, significantly complicated the task of kernel programming. Device driver programmers must now factor concurrency into their designs from the beginning, and they must have a strong understanding of the facilities provided by the kernel for concurrency management.

The purpose of this chapter is to begin the process of creating that understanding. To that end, we introduce facilities that are immediately applied to the *scull* driver from Chapter 3. Other facilities presented here are not put to use for some time yet. But first, we take a look at what could go wrong with our simple *scull* driver and how to avoid these potential problems.

Pitfalls in scull

Let us take a quick look at a fragment of the *scull* memory management code. Deep down inside the *write* logic, *scull* must decide whether the memory it requires has been allocated yet or not. One piece of the code that handles this task is:

```
if (!dptr->data[s_pos]) {
    dptr->data[s_pos] = kmalloc(quantum, GFP_KERNEL);
    if (!dptr->data[s_pos])
        goto out;
}
```

Suppose for a moment that two processes (we'll call them "A" and "B") are indepen dently attempting to write to the same offset within the same *scull* device. Each process reaches the `i f` test in t he first l ine of the fragment above at the same time. If the pointer in question is `NULL`, each process will decide to allocate memory, and each will assign the resulting pointer to `dptr->data[s_pos]`. Since both processes are assigning to the same location, clearly only one of the assignments will prevail.

What will happen, of course, is that the process that completes the assignment second will "win." If process A assigns first, its assignment will be overwritten by process B. At that point, *scull* will forget entirely about the memory that A allocated; it only has a pointer to B's memory. The memory allocated by A, thus, will be dropped and never returned to the system.

This sequence of events is a demonstration of a *race condition* . Race conditions are a result of uncontrolled access to shared data. When the wrong access pattern happens, something unexpected results. For the race condition discussed here, the result is a memory leak. That is bad enough, but race conditions can often lead to system crashes, corrupted data, or security problems as well. Programmers can be tempted to disregard race conditions as extremely low probability events. But, in the computing world, one-in-a-million events can happen every few seconds, and the consequences can be grave.

We will eliminate race conditions from *scull* shortly, but first we need to take a more general view of concurrency.

Concurrency and Its Management

In a modern Linux system, there are numerous s ources of concurrency and, therefore, possible race conditions. Multiple user-space processes are running, and they can access your code in surprising combinations of ways. SMP systems can be executing your code simultaneously on different processors. Kernel code is preemptible; your driver's code can lose the processor at any time, and the process that replaces it could also be running in your driver. Device interrupts are asynchronous events that can cause concurrent execution of your code. The kernel also provides various mechanisms for delayed code execution, such as workqueues, tasklets, and timers, which can cause your code to run at any time in ways unrelated to what the current process is doing. In the modern, hot-pluggable world, your device could simply disappear while you are in the middle of working with it.

Avoidance of race conditions can be an intimidating task. In a world where anything can happen at any time, how does a driver pro grammer av oid the cr eation of absolute c haos? As it turns out, most race conditions can be avoided through some thought, the kernel's concurrency control primitives, and the application of a few basic principles. We'll start with the principles first, then get into the specifics of how to apply them.

Race conditions come about as a result of shared access to resources. When two threads of execution[1] have a reason to work with the same data structures (or hardware resources), the potential for mixups always exists. So the first rule of thumb to keep in mind as you design your driver is to avoid shared resources whenever possible. If there is no concurrent access, there can be no race conditions. So carefully-written kernel code should have a minimum of sharing. The most obvious application of this idea is to avoid the use of global variables. If you put a resource in a place where more than one thread of execution can find it, there should be a strong reason for doing so.

The fact of the matter is, however, that such sharing is often required. Hardware resources are, by their nature, shared, and software resources also must often be available to more than one thread. Bear in mind as well that global variables are far from the only way to share data; any time your code passes a pointer to some other part of the kernel, it is potentially creating a new sharing situation. Sharing is a fact of life.

Here is the hard rule of resource sharing: any time that a hardware or software resource is shared beyond a single thread of execution, and the possibility exists that one thread could encounter an inconsistent view of that resource, you must explicitly manage access to that resource. In the *scull* example above, process B's view of the situation is inconsistent; unaware that process A has already allocated memory for the (shared) device, it performs its own allocation and overwrites A's work. In this case, we must control access to the *scull* data structure. We need to arrange things so that the code either sees memory that

has been allocated or knows that no memory has been *or will be* allocated by anybody else. The usual technique for access management is called *locking* or *mutual exclusion* — making sure that only one thread of execution can manipulate a shared resource at any time. Much of the rest of this chapter will be devoted to locking.

First, however, we must briefly consider one other important rule. When kernel code creates an object that will be shared with any other part of the kernel, that object must continue to exist (and function properly) until it is known that no outside references to it exist. The instant that *scull* makes its devices available, it must be prepared to handle requests on those devices. And *scull* must continue to be able to handle requests on its devices until it knows that no reference (such as open user-space files) to those devices exists. Two requirements come out of this rule: no object can be made available to the kernel until it is in a state where it can function properly, and refere nces to su ch objects must be t racked. In most cases, you'll find that the kernel handles reference counting for you, but there are always exceptions.

Following the above rules requires planning and careful attention to detail. It is easy to be surprised by concurrent access to resources you hadn't realized were shared. With some effort, however, most race conditions can be headed off before they bite you — or your users.

Semaphores and Mutexes

So let us look at how we can add locking to *scull*. Our goal is to make ou r operations on the *scull* data structure *atomic*, meaning that the entire operation happens at once as far as other threads of execution are concerned. For our memory leak example, we need to ensure that if one thread finds that a particular chunk of memory must be allocated, it has the opportunity to perform that allocation before any other thread can make that test. To this end, we must set up *critical sections*: code that can be executed by only one thread at any given time.

Not all critical sections are the same, so the kernel provides different primitives for different needs. In this case, every access to the *scull* data structur e happens in process context as a result of a direct user request; no accesses will be made from interrupt handlers or other asynchronous contexts. There are n o particul ar latency (response time) req uirements; applicati on program mers understand that I/O requests are not usually satisfied immediately. Furthermore, the *scull* is not holding any other critical system resource while it is accessing its own data structures. What all this means is that if the *scull* driver goes to sleep while waiting for its turn to access the data structure, nobody is going to mind.

"Go to sleep" is a well-defined term in this context. When a Linux process reaches a point where it cannot make any further processes, it goes to sleep (or "blocks"), yielding the processor to somebody else until some future t ime when i t can get work done again. Processes often sleep when waiting for I/O to complete. As we get deeper into the kernel, we will encounter a number of situations where we cannot sleep. The *write* method in *scull* is not one of those situations, however. So we can use a locking mechanism that might cause the process to sleep while waiting for access to the critical section.

Just as importantly, we will be performing an operation (memory allocation with *kmalloc*) that could sleep — so sleeps are a possibility in any case. If our critical sections are to work properly, we must use a locking primitive that works when a thread that owns the lock sleeps. Not all locking mechanisms can be used where sleeping is a possibility (we'll see some that don't later in this chapter). For our present needs, however, the mechanism that fits best is a *semaphore*.

Semaphores are a well-understood concept in computer science. At its core, a semaphore is a single integer value combined with a pair of functions that are typically called *P* and *V*. A process wishing to enter a critical section will call *P* on the relevant semaphore; if the semaphore's value is greater than zero, that value is decremented by one and the process continues. If, instead, the semaphore's value is `0` (or less), the process must wait until somebody else releases the semaphore. Unl ocking a semaphore is accomplished by calling *V*; this function increments the value of the semaphore and, if necessary, wakes up processes that are waiting.

When semaphores are used for *mutual exclusion* — keeping multiple processes from running within a critical section simultaneously — their value will be initially set to `1`. Such a semaphore can be held only by a single process or thread at any given time. A semaphore used in this mode is sometimes called a *mutex*, which is, of course, an abbreviation for "mutual exclusion." Almost all semaphores found in the Linux kernel are used for mutual exclusion.

The Linux Semaphore Implementation

The Linux kernel provides an implementation of semaph ores that conforms to the above semantics, although the terminology is a little different. To use semaphores, kernel code must include *<asm/semaphore.h>*. The relevant type is `struct semaphore`; actual semaphores can be declared and initialized in a few ways. One is to create a semaphore directly, then set it up with *sema_init*:

```
void sema_init(struct semaphore *sem, int val);
```

where `val` is the initial value to assign to a semaphore.

Usually, however, semaphores are used in a mutex mode. To make this common case a little easier, the kernel has provided a set of helper functions an d macros. Thus, a mu tex can be declared and initialized with one of the following:

```
DECLARE_MUTEX(name);
DECLARE_MUTEX_LOCKED(name);
```

Here, the result is a semaphore variable (called `name`) that is initialized to `1` (with `DECLARE_MUTEX`) or `0` (with `DECLARE_MUTEX_LOCKED`). In the latter case, the mutex starts out in a locked state; it will have to be explicitly unlocked before any thread will be allowed access.

If the mutex must be initialized at runtime (which is the case if it is allocated dynamically, for example), use one of the following:

```
void init_MUTEX(struct semaphore *sem);
void init_MUTEX_LOCKED(struct semaphore *sem);
```

In the Linux world, the *P* function is called *down* — or some variation of that name. Here, "down" refers to the fact that the function decrements the value of the semaphore and, perhaps after putting the caller to sleep for a while to wait for the semaphore to become available, grants a ccess to t he protected resources. There are three versions of *down*:

```
void down(struct semaphore *sem);
int down_interruptible(struct semaphore *sem);
int down_trylock(struct semaphore *sem);
```

down decrements the value of the semaphore and waits as long as need be. *down_interruptible* does the same, but the operation is interruptible. The interruptible version is almost always the one you will want; it allows a user-space process that is waiting on a semaphore to be interrupted by the user. You do not, as a general rule, want to use noninterruptible operations unless there truly is no alternative. Non-interruptible operations are a good way to create unkillable processes (the dreaded "D state" seen in *ps*), and annoy your users. Using *down_interruptible* requires some extra care, however, if the operation is interrupted, the function returns a nonzero value, and the caller does *not* hold the semaphore. Proper use of *down_interruptible* requires always checking the return value and responding accordingly.

The final version (*down_trylock*) never sleeps; if the semaphore is not available at the tim e of the call, *down_trylock* returns immediately with a nonzero return value.

Once a thread has successfully called one of the versions of *down*, it is said to be "holding" the semaphore (or to have "taken out" or "acquired" the semaphore). That thread is now entitled to access the critical section protected by the semaphore. When the operations requiring mutual exclusion are complete, the semaphore must be returned. The Linux equivalent to *V* is *up*:

```
void up(struct semaphore *sem);
```

Once *up* has been called, the caller no longer holds the semaphore.

As you would expect, any thread that takes out a semaphore is required to release it with one (and only one) call to *up*. Special care is often required in error paths; if an error is encountered while a semaphore is held, that semaphore must be released before returning the error status to the caller. Failure to free a semaphore is an easy error to make; the result (processes hanging in seemingly unrelated places) can be hard to reproduce and track down.

Using Semaphores in scull

The semaphore mechanism gives *scull* a tool that can be used to avoid race conditions while accessing the `scull_dev` data structure. But it is up to us to use that tool correctly. The keys to proper us e of locki ng primitives are to specify exactly which resources are to be protected and to make sure that every access to those resources uses the proper locking. In our example driver, everything of interest is contained within the `scull_dev` structure, so that is the logical scope for our locking regime.

Let's look again at that structure:

```
struct scull_dev {
    struct scull_qset *data;  /* Pointer to first quantum set */
    int quantum;               /* the current quantum size */
    int qset;                 /* the current array size */
    unsigned long size;       /* amount of data stored here */
    unsigned int access_key;  /* used by sculluid and scullpriv */
    struct semaphore sem;     /* mutual exclusion semaphore     */
    struct cdev cdev;     /* Char device structure      */
};
```

Toward the bottom of the structure is a member called `sem` which is, of course, our semaphore. We have chosen to use a separate semaphore for each virtual *scull* device. It would have been equally correct to use a single, global semaphore. The various *scull* devices share no resources in common, however, and there is no reason to make one process wait while another process is working with a different *scull* device. Using a separate semaphore for each device allows operations on different devices to proceed in parallel and, therefore, improves performance.

Semaphores must be initialized before use. *scull* performs this initialization at load time in this loop:

```
    for (i = 0; i < scull_nr_devs; i++) {
        scull_devices[i].quantum = scull_quantum;
        scull_devices[i].qset = scull_qset;
        init_MUTEX(&scull_devices[i].sem);
        scull_setup_cdev(&scull_devices[i], i);
    }
```

Note that the semaphore must be initialized *before* the *scull* device is made available to the rest of the system. Therefore, *init_MUTEX* is called before *scull_setup_cdev*. Performing these operations in the opposite order would create a race condition where the semaphore could be accessed before it is ready.

Next, we must go through the code and make sure that no accesses to the `scull_dev` data structure are made without holding the semaphore. Thus, for example, *scull_write* begins with this code:

```
    if (down_interruptible(&dev->sem))
        return -ERESTARTSYS;
```

Note the check on the return value of *down_interruptible*; if it returns nonzero, the operation was interrupted. The usual thing to do in this situation is to return `-ERESTARTSYS`. Upon seeing this return code, the higher layers of the kernel will either restart the call from the beginning or return the error to the user. If you return `-ERESTARTSYS`, you must first undo any user-visible changes that might have been made, so that the right thing happens when the system call is retried. If you cannot undo things in this manner, you should return `-EINTR` instead.

scull_write must release the semaphore whether or not it was able to carry out its other tasks successfully. If all goes well, execution falls into the final few lines of the function:

```
out:
  up(&dev->sem);
  return retval;
```

This code frees the semaphore and returns whatever status is called for. There are several places in *scull_write* where things can go wrong; these include memory allocation failures or a fault while trying to copy data from user space. In those cases, the code performs a `goto out`, ensuring that the proper cleanup is done.

Reader/Writer Semaphores

Semaphores perform mutual exclusion for all callers, regardless of what each thread may want to do. Many tasks break down into two distinct types of work, however: tasks that only need to read the protected data structures and those that must make changes. It is often possible to allow multiple concurrent readers, as long as nobody is trying to make any changes. Doing so can optimize performance significantly; read-only tasks can get their work done in parallel without having to wait for other readers

to exit the critical section.

The Linux kernel provides a special type of semaphore called a *rwsem* (or “reader/writer semaphore”) for this situation. The use of rwsems in drivers is relatively rare, but they are occasionally useful.

Code using rwsems must include *<linux/rwsem.h>*. The relevant data type for reader/writer semaphores is `struct rw_semaphore`; an rwsem must be ex plicitly i nitialized at runtime with:

```
void init_rwsem(struct rw_semaphore *sem);
```

A newly initialized rwsem is available for the next task (reader or writer) that comes along. The interface for code needing read-only access is:

```
void down_read(struct rw_semaphore *sem);
int down_read_trylock(struct rw_semaphore *sem);
void up_read(struct rw_semaphore *sem);
```

A call to *down_read* provides read-only access to the protected resources, possibly concurrently with other readers. Note that *down_read* may put the calling process into an uninterruptible sleep. *down_read_trylock* will not wait if read access is unavailable; it returns nonzero if access was granted, `0` otherwise. Note that the convention for *down_read_trylock* differs from that of most kernel functions, where success is indicated by a return value of `0`. A rwsem obtained with *down_read* must eventually be freed with *up_read*.

The interface for writers is similar:

```
void down_write(struct rw_semaphore *sem);
int down_write_trylock(struct rw_semaphore *sem);
void up_write(struct rw_semaphore *sem);
void downgrade_write(struct rw_semaphore *sem);
```

down_write, *down_write_trylock*, and *up_write* all behave just like their reader counterparts, except, of course, that they provide write access. If you have a situation where a writer lock is needed for a quick change, followed by a longer period of read-only access, you can use *downgrade_write* to allow other readers in once you have finished making changes.

An rwsem allows eithe r one writer or an unlimited number of readers to hold the semaphore. Writers get priority; as soon as a writer tries to enter the critical section, no readers will be allowed in until all writers have completed their work. This implementation can lead to reader *starvation* — where readers are denied access for a long time — if you have a large number of writers contending for the semaphore. For this reason, rwsems are best used when write access is required only rarely, and writer access is held for short periods of time.

Completions

A common pattern in kernel programming involves initiating some activity outside of the current thread, then waiting for that activity to complete. This activity can be the creation of a new kernel thread or user-space process, a request to an existing process, or some sort of hardware-based action. It such cases, it can be tempting to use a semaphore for synchronization of the two tasks, with code such as:

```
struct semaphore sem;

init_MUTEX_LOCKED(&sem);
start_external_task(&sem);
down(&sem);
```

The external task can then call `up(& sem)` when its work is done.

As is turns out, semaphores are not the best tool to use in this situation. In normal use, code attempting to lock a semaphore finds that semaphore available almost all the time; if there is significant contention for the semaphore, pe rformance suffers an d the lock ing scheme needs to be reviewed. So semaphores have been heavily optimized for the “available” case. When used to communicate task completion in the way shown above, however, the thread calling *down* will almost always have to wait; performance will suffer accordingly. Semaphores can also be subject to a (difficult) race condition wh en used in this way if they are declared as automatic variables. In some cases, the semaphore could vanish before the process calling *up* is finished with it.

These concerns inspired the addition of the “completion” interface in the 2.4.7 kernel. Completions are a lightweight mechanism with one task: allowing one thread to tell another that the job is done. To use completions, your code must include *<linux/completion.h>*. A completion can be created with:

```
DECLARE_COMPLETION(my_completion);
```

Or, if the completion must be created and initialized dynamically:

```
struct completion my_completion;
/* ... */
init_completion(&my_completion);
```

Waiting for the completion is a simple matter of calling:

```
void wait_for_completion(struct completion *c);
```

Note that this function performs an uninterruptible wait. If your code calls *wait_for_completion* and nobody ever completes the task, the result will be an unkillable process.[2]

On the other side, the actual completion event may be signalled by calling one of the following:

```
 void complete(struct completion *c);
void complete_all(struct completion *c);
```

The two functions behave differently if more than one thread is waiting for the same completion event. *complete* wakes up only one of the waiting threads while *complete_all* allows all of them to proceed. In most cases, there is only one waiter, and the two functions will produce an identical result.

A completion is normally a one-shot device; it is used once then discarded. It is possible, however, to reuse completion structures if proper care is taken. If *complete_all* is not used, a completion structure can be reuse d without any problems as long as there is no ambiguity about what event is being signalled. If you use *complete_all*, however, you must reinitialize the completion structure before reusing it. The macro:

```
INIT_COMPLETION(struct completion c);
```

can be used to quickly perform this reinitialization.

As an example of how completions may be used, consider the *complete* module, which is included in the example source. This module defines a device with simple semantics: any process trying to read from the device will wait (using *wait_for_completion*) until some other process writes to the device. The code which implements this behavior is:

```
DECLARE_COMPLETION(comp);

ssize_t complete_read (struct file *filp, char _ _user *buf, size_t count, loff_t *pos)
{
    printk(KERN_DEBUG "process %i (%s) going to sleep\n",
           current->pid, current-> ;comm);
    wait_for_completion(&comp);
    printk(KERN_DEBUG "awoken %i (%s)\n", current->pid, current->comm);
    return 0; /* EOF */
}

ssize_t complete_write (struct file *filp, const char _ _user *buf, size_t count,
        loff_t *pos)
{
    printk(KERN_DEBUG "process %i (%s) awakening the readers...\n",
 & nbsp;          current->pid, current->comm);
    complete(&comp);
    return count; /* succeed, to avoid retrial */
}
```

It is possible to have multiple processes “reading” from this device at the same time. Each write to the device will cause exactly one read operation to complete, but there is no way to know which one it will be.

A typical use of the completion mechanism is with kernel thread termination at module exit time. In the prototypical case, some of the driver internal workings is performed by a kernel thread in a `while (1)` loop. When the module is ready to be cleaned up, the exit function tells the thread to exit and then waits for completion. To this aim, the kernel includes a specific function to be used by the thread:

```
void complete_and_exit(struct completion *c, long retval);
```

Spinlocks

Semaphores are a useful tool for mutual exclusion, but they are not the only such tool provided by the kernel. Instead, most locking is implemented with a mechanism called a *spinlock*. Unlike semaphores, spinlocks may be used in code that cannot sleep, such as interrupt handlers. When properly used, spinlocks offer higher performance than semaphores in general. They do, however, bring a different set of constraints on their use.

Spinlocks are simple in concept. A spinlock is a mutual exclusion device that can have only two values: “locked” and “unlocked.” It is usually implemented as a single bit in an i nteger val ue. Code w ishing to take out a particular lock tests the relevant bit. If the lock is available, the “locked” bit is set and the code continues into the critical section. If, instead, the lock has been taken by somebody else, the code goes into a tight loop where it repeatedly checks the lock until it becomes available. This loop is the “spin” part of a spinl ock.

Of course, the real implementation of a spinlock is a bit more complex than the description above. The “test an d set” operation must be done in an atomic manner so that only one thread can obtain the lock, even if several are spinning at any given time. Care must also be taken to avoid deadlocks on *hyperthreaded* processors — chips that implement multiple, virtual CPUs sharing a single processor core and cache. So the actual spinlock implementation is different for every architecture that Linux supports. The core concept is the same on all systems, however, when there is contention for a spinlock, the processors that are waiting execute a tight loop and accomplish no useful work.

Spinlocks are, by their nature, intended for use on multiprocessor systems, although a uniprocessor workstation running a preemptive kernel behaves like SMP, as far as concurrency is concerned. If a nonpreemptive uniprocessor system ever went into a spin on a lock, it would spin forever; no other thread would ever be able to obtain the CPU to release the lock. For this reason, spinlock operations on uniprocessor systems without preemption enabled are optimized to do nothing, with the exception of the ones that change the IRQ masking status. Because of preemption, even if you never expect your code to run on an SMP system, you still need to implement proper locking.

Introduction to the Spinlock API

The required include file for the spinlock primitives is *<linux/spinlock.h>*. An actual lock has the type `spinlock_t`. Like any other data structure, a spinlock must be initialized. This initialization may be done at compile time as follows:

```
spinlock_t my_lock = SPIN_LOCK_UNLOCKED;
```

or at runtime with:

```
void spin_lock_init(spinlock_t *lock);
```

Before entering a critical section, your code must obtain the requisite lock with:

```
void spin_lock(spinlock_t *lock);
```

Note that all spinlock waits are, by their nature, uninterruptible. Once you call *spin_lock* , you will spin until the lock becomes available.

To release a lock that you have obtained, pass it to:

```
void spin_unlock(spinlock_t *lock);
```

There are many other spinlock functions, and we will look at them all shortly. But none of them depart from the core idea shown by the functions listed above. There is very little that one can do with a lock, other than lock and release it. However, there are a few rules about how you must work with spinlocks. We will take a moment to look at those before getting into the full spinlock interface.

Spinlocks and Atomic Context

Imagine for a moment that your driver acquires a spinlock and goes about its business within its critical section. Somewhere in the middle, your driver loses the processor. Perhaps it has called a function (*copy_from_user*, say) that puts the process to sleep. Or, perhaps, kernel preemption kicks in, and a higher-priority process pushes your code aside. Your code is now holding a lock that it will not release any time in the foreseeable future. If some other thread tries to obtain the same lock, it will, in the best case, wait (spinning in the processor) for a very long time. In the worst case, the system could deadlock entirely.

Most readers would agree that this scenario is best avoided. Therefore, the core rule that applies to spinlocks is that any code must, while holding a spinlock, be atomic. It cannot sleep; in fact, it cannot relinquish the processor for any reason except to service interrupts (and sometimes not even then).

The kernel preemption case is handled by the spinlock code itself. Any time kernel code holds a spinlock, preemption is disabled on the relevant processor. Even uniprocessor systems must disable preemption in this way to avoid race conditions. That is why proper locking is required even if you never expect your code to run on a multiprocessor machine.

Avoiding sleep while holding a lock can be more difficult; many kernel functions can sleep, and this behavior is not always well documented. Copying data to or from user space is an obvious example: the required user-space page may need to be swapped in from the disk before the copy can proceed, and that operation clearly requires a sleep. Just about any operation that must allocate memory can sleep; *kmalloc* can decide to give up the processor, and wait for more memory to become available unless it is explicitly told not to. Sleeps can happen in surprising places; writing code that will execute under a spinlock requires paying attention to every function that you call.

Here's another scenario: your driver is executing and has just taken out a lock that controls access to its device. While the lock is held, the device issues an interrupt, which causes your interrupt handler to run. The interrupt handler, before accessing the device, must also obtain the lock. Taking out a spinlock in an interrupt handler is a legitimate thing to do; that is one of the reasons that spinlock operations do not sleep. But what happens if the interrupt routine executes in the same processor as the code that took out the lock originally? While the interrupt handler is spinning, the noninterrupt code will not be able to run to release the lock. That processor will spin forever.

Avoiding this trap requires disabling interrupts (on the local CPU only) while the spinlock is held. There are variants of the spinlock functions that will disable interrupts for you (we'll see them in the next section). However, a complete discussion of interrupts must wait until Chapter 10.

The last important rule for spinlock usage is that spinlocks must always be held for the minimum time possible. The longer you hold a lock, the longer another processor may have to spin waiting for you to release it, and the chance of it having to spin at all is greater. Long lock hold times also keep the current processor from scheduling, meaning that a higher priority process — which really should be able to get the CPU — may have to wait. The kernel developers put a great deal of effort into reducing kernel latency (the time a process may have to wait to be scheduled) in the 2.5 development series. A poorly written driver can wipe out all that progress just by holding a lock for too long. To avoid creating this sort of problem, make a point of keeping your lock-hold times short.

The Spinlock Functions

We have already seen two functions, *spin_lock* and *spin_unlock*, that manipulate spinlocks. There are several other functions, however, with similar names and purposes. We will now present the full set. This discussion will take us into ground we will not be able to cover properly for a few chapters yet; a complete understanding of the spinlock API requires an understanding of interrupt handling and related concepts.

There are actually four functions that can lock a spinlock:

```
void spin_lock(spinlock_t *lock);
void spin_lock_irqsave(spinlock_t *lock, unsigned long flags);
void spin_lock_irq(spinlock_t *lock);
void spin_lock_bh(spinlock_t *lock)
```

We have already seen how *spin_lock* works. *spin_lock_irqsave* disables interrupts (on the local processor only) before taking the spinlock; the previous interrupt state is stored in `flags`. If you are absolutely sure nothing else might have already disabled interrupts on your processor (or, in other words, you are sure that you should enable interrupts when you release your spinlock), you can use *spin_lock_irq* instead and not have to keep track of the flags. Finally, *spin_lock_bh* disables software interrupts before taking the lock, but leaves hardware interrupts enabled.

If you have a spinlock that can be taken by code that runs in (hardware or software) interrupt context, you must use one of the forms of *spin_lock* that disables interrupts. Doing otherwise can deadlock the system, sooner or later. If you do not access your lock in a hardware interrupt handler, but you do via software interrupts (in code that runs out of a tasklet, for example, a topic covered in Chapter 7), you can use *spin_lock_bh* to safely avoid deadlocks while still allowing hardware interrupts to be serviced.

There are also four ways to release a spinlock; the one you use must correspond to the function you used to take the lock:

```
void spin_unlock(spinlock_t *lock);
void spin_unlock_irqrestore(spinlock_t *lock, unsigned long flags);
void spin_unlock_irq(spinlock_t *lock);
void spin_unlock_bh(spinlock_t *lock);
```

Each *spin_unlock* variant undoes the work performed by the corresponding *spin_lock* function. The `flags` argument passed to *spin_unlock_irqrestore* must be the same variable passed to *spin_lock_irqsave*. You must also call *spin_lock_irqsave* and *spin_unlock_irqrestore* in the same function; otherwise, your code may break on some architectures.

There is also a set of nonblocking spinlock operations:

```
int spin_trylock(spinlock_t *lock);
int spin_trylock_bh(spinlock_t *lock);
```

These functions return nonzero on success (the lock was obtained), `0` otherwise. There is no "try" version that disables interrupts.

Reader/Writer Spinlocks

The kernel provides a reader/writer form of spinlocks that is directly analogous to the reader/writer semaphores we saw earlier in this chapter. These locks allow any number of readers into a critical section simultaneously, but writers must have exclusive access. Reader/writer locks have a type of `rwlock_t`, defined in *<linux/spinlock.h>*. They can be declared and initialized in two wa ys:

```
rwlock_t my_rwlock = RW_LOCK_UNLOCKED; /* Static way */

rwlock_t my_rwlock;
rwlock_init(&my_rwlock);  /* Dynamic way */
```

The list of functions available should look reasonably familiar by now. For readers, the following functions are available:

```
void read_lock(rwlock_t *lo ck);
void read_lock_irqsave(rwlock_t *lock, unsigned long flags);
void read_lock_irq(rwlock_t *lock);
void read_lock_bh(rwlock_t *lock);

void read_unlock(rwlock_t *lock);
void read_unlock_irqrestore(rwlock_t *lock, unsigned long flags);
void read_unlock_irq(rwlock_t *lock);
void read_unlock_bh(rwlock_t *lock);
```

Interestingly, there is no *read_trylock*.

The functions for write access are similar:

```
void write_lock(rwlock_t *lock);
void write_lock_irqsave(rwlock_t *lock, unsigned long flags);
void write_lock_irq(rwlock_t *lock);
void write_lock_bh(rwlock_t *lock);
int write_trylock(rwlock_t *lock);

void write_unlock(rwlock_t *lock);
void write_unlock_irqrestore(rwlock_t *lock, unsigned long flags);
void write_unlock_irq(rwlock_t *lock);
void write_unlock_bh(rwlock_t *lock);
```

Reader/writer locks can starve readers just as rwsems can. This behavior is rarely a problem; however, if there is enough lock contention to bring about starvation, performance is poor anyway.

Locking Traps

Many years of experience with locks — experience that predates Linux — have shown that locking can be very hard to get right. Managing concurrency is an inherently tricky undertaking, and there are many ways of making mistakes. In this section, we take a quick look at things that can go wrong.

Ambiguous Rules

As has already been said above, a proper locking scheme requires clear and explicit rules. When you create a resource that can be accessed concurrently, you should define which lock will control that access. Locking should r eally be laid out at the beginning; it can be a hard thing to retrofit in afterward. Time taken at the outset usually is paid back generously at debugging time.

As you write your code, you will doubtless encounter several functions that all require access to structures protected by a specific lock. At this point, you must be c areful: if one funct ion acquir es a lock and then calls another function that also attempts to acquire the lock, your code deadlocks. Neither semaphores nor spinlocks allow a lock holder to acquire the lock a second time; should you attempt to do so, things simply hang.

To make your locking work properly, you have to write some functions with the assumption that their caller has already acquired the relevant lock(s). Usually, only your internal, static functions can be written in this way; functions called from outside must handle locking explicitl y. When you write internal functions that make assumptions about locking, do yourself (and anybody else who works with your code) a favor and document those assumptions explicitly. It can be very hard to come back months later and figure out whether you need to hold a lock to call a particular function or not.

In the case of *scull*, the design decision taken was to require all functions invoked directly from sy stem calls to acquir e the sema phore applying to the device structure that is accessed. All internal functions, which are only called from other *scull* functions, can then assume that the semaphore has been properly acquired.

Lock Ordering Rules

In systems with a large number of locks (and the kernel is becoming such a system), it is not unusual for code to need to hold more than one lock at once. If some sort of computation must be performed using two different resources, each of which has its own lock, there is often no alternative to acquiring both locks.

Taking multiple locks can be dangerous, however. If you have two locks, called *Lock1* and *Lock2*, and code needs to acquire both at the same time, you have a potential deadlock. Just imagine one thread locking *Lock1* while another simultaneously takes *Lock2*. Then each thread tries to get the one it doesn’t have. Both threads will deadlock.

The solution to this problem is usually simple: when multiple locks must be acquired, they should always be acquired in the same order. As long as this convention i s followed, simple deadlocks like the one described above can be avoided. However, following lock ordering rules can be easier said than done. It is ver y rare tha t such rul es are actually written down anywhere. Often the best you can do is to see what other code does.

A couple of rules of thumb can help. If you must obtain a lock that is local to your code (a device lock, say) along with a lock belonging to a more central part of the kernel, take your lock first. If you have a combination of semaphores and spinlocks, you must, of course, obtain the semaphore(s) first; calling *down* (which can sleep) while holding a spinlock is a serious error. But

most of all, try to avoid situations where you need more than one lock.

Fine- Versus Coarse-Grained Locking

The first Linux kernel that supported multiprocessor systems was 2.0; it contained exactly one spinlock. The *big kernel lock* turned the entire kernel into one large critical section; only one CPU could be executing kernel code at any given time. This lock solved the concurrency problem well enough to allow the kernel developers to address all of the other issues involved in supporting SMP. But it did not scale very well. Even a two-processor system could spend a significant amount of time simply waiting for the big kernel lock. The performance of a four-processor system was not even close to that of four independent machines.

So, subsequent kernel releases have included finer-grained locking. In 2.2, one spinlock controlled access to the block I/O subsystem; another worked for networking, and so on. A modern kernel can contain thousands of locks, each protecting one small resource. This sort of fine-grained locking can be good for scalability; it allows each processor to work on its specific task without contending for locks used by other processors. Very few people miss the big kernel lock.[3]

Fine-grained locking comes at a cost, however. In a kernel with thousands of locks, it can be very hard to know which locks you need — and in which order you should acquire them — to perform a specific operation. Remember that locking bugs can be very difficult to find; more locks provide more opportunities for truly nasty locking bugs to creep into the kernel. Fine-grained locking can bring a level of complexity that, over the long term, can have a large, adverse effect on the maintainability of the kernel.

Locking in a device driver is usually relatively straightforward; you can have a single lock that covers everything you do, or you can create one lock for every device you manage. As a general rule, you should start with relatively coarse locking unless you have a real reason to believe that contention could be a problem. Resist the urge to optimize prematurely; the real performance constraints often show up in unexpected places.

If you do suspect that lock contention is hurting performance, you may find the *lockmeter* tool useful. This patch (available at http://oss.sgi.com/projects/lockmeter/) instruments the kernel to measure time spent waiting in locks. By looking at the report, you are able to determine quickly whether lock contention is truly the problem or not.

Alternatives to Locking

The Linux kernel provides a number of powerful locking primitives that can be used to keep the kernel from tripping over its own feet. But, as we have seen, the design and implementation of a locking scheme is not without its pitfalls. Often there is no alternative to semaphores and spinlocks; they may be the only way to get the job done properly. There are situations, however, where atomic access can be set up without the need for full locking. This section looks at other ways of doing things.

Lock-Free Algorithms

Sometimes, you can recast your algorithms to avoid the need for locking altogether. A number of reader /writer si tuations — if there is only on e writer — can often work in this manner. If the writer takes care that the view of the data structure, as seen by the reader, is always consiste nt, it may be possible to create a lock-free data structure.

A data structure that can often be useful for lockless p roducer/co nsumer tas ks is the *circular buffer* . This algorithm involves a producer placing data into one end of an array, while the consumer removes data from the other. When the end of the array is reached, the producer wraps back around to the beginning. So a circular buffer requires an array and two index values to track where the next new value goes and which value should be removed from the buffer next.

When carefully implemented, a circular buffer requires no locking in the absence of multiple producers or consumers. The producer is the only thread that is allowed to modify the write index and the array location it points to. As long as the writer stores a new value into th e buffer b efore updating the write index, the reader will always see a consistent view. The reader, in turn, is the only thread that can access the read index and the value it points to. With a bit of care to ensure that the two pointers do not overrun each other, the producer and the consumer can access the buffer concurrently with no race conditions.

Figure 5-1 shows circular buffer in several states of fill. This buff er has bee n defined such that an empty condition is indicated by the read and write pointers being equal, while a full condition happens whenever the write pointer is immediately behind the read pointer (being careful to account for a wrap!). When carefully programmed, this buffer can be used without locks.

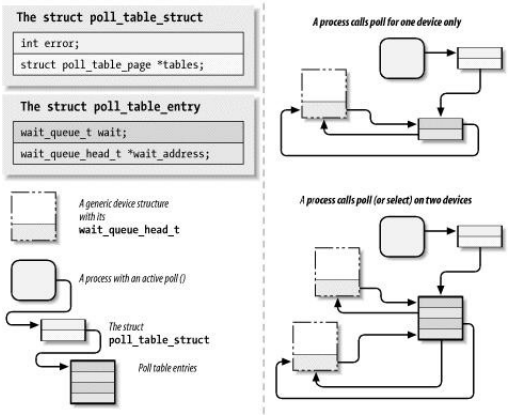

Figure 5-1. A circular buffer

Circular buffers show up reasonably often in device drivers. Networking adaptors, in particular, often use circular buffers to exchange data (packets) with the processor. Note that, as of 2.6.10, there is a generic circular buffer implementation available in the kernel; see *<linux/kfifo.h>* for information on how to use it.

Atomic Variables

Sometimes, a shared resource is a simple integer value. Suppose your driver maintains a shared variable `n_op` that tells how many device operations are currently outstanding. Normally, even a simple operation such as:

```
n_op++;
```

would require locking. Some processors might perform that sort of increment in an atomic manner, but you can't count on it. But a full locking regime seems like overhead for a simple integer value. For cases like this, the kernel provides an atomic integer type called `atomic_t`, defined in *<asm/atomic.h>*.

An `atomic_t` holds an `int` value on all supported architectures. Because of the way this type works on some processors, however, the full integer range may not be available; thus, you shoul d not coun t on an `atomic_t` holding more than 24 bits. The following operations are defined for the type and are guaranteed to be atomic with respect to all processors of an SMP computer. The operations are very fast, because they compile to a single machine instruction whenever possible.

```
void atomic_set(atomic_t *v, int i);
atomic_t v = ATOMIC_INIT(0);
```

Set the atomic variable `v` to the integer value `i`. You can also initialize atomic values at compile time with the `ATOMIC_INIT` macro.

```
int atomic_read(atomic_t *v);
```

Return the current value of `v`.

```
void atomic_add(int i, atomic_t *v);
```

Add `i` to the atomic variable pointed to by `v`. The return value is `void`, because there is an extra cost to returning the new value, and most of the time there's no need to know it.

```
void atomic_sub(int i, atomic_t *v);
```

Subtract `i` from `*v`.

```
void atomic_inc(atomic_t *v);
    void atomic_dec(atomic_t *v);
```

Increment or decrement an atomic variable.

```
 int  atomic_in c_and_test(atomic_t *v);
int atomic_dec_and_test(atomic_t *v);
int atomic_sub_and_test(int i, atomic_t *v);
```

Perform the specified operation and test the result; if, after the operation, the atomic value is `0`, then the return value is true; otherwise, it is false. Note that there is no *atomic_add_and_test*.

```
int atomic_add_negative(int i, atomic_t *v);
```

Add the integer variable `i` to `v`. The return val ue is true if the result is negative, false otherwise.

```
int atomic_add_return(int i, atomic_t *v);
int atomic_sub_return(int i, atomic_t *v);
int atomic_inc_return(atomic_t *v);
    int atom ic_dec_ret urn(atomic _t *v);
```

Behave just like *atomic_add* and friends, with the exception that they return the new value of the atomic variable to the caller.

As stated earlier, `atomic_t` data items must be accessed only through these functions. If you pass an atomic item to a function that expects an integer argument, you'll get a compiler error.

You should also bear in mind that `atomic_t` values work only when the quantity in question is truly atomic. Operations requiring multiple `atomic_t` variables still require some other sort of locking. Consider the following code:

```
atomic_sub(amount, &first_atomic);
atomic_add(amount, &second_atomic);
```

There is a period of time where the `amount` has been subtracted from the first atomic value but not yet added to the second. If that state of affairs could create trouble for code that might run between the two operations, some form of locking must be employed.

Bit Operations

The `atomic_t` type is good for performing integer arithmetic. It doesn't work as well, however, when you need to manipulate individual bits in an atomic manner. For that purpose, instead, the kernel offers a set of functions that modify or test single bits atomically. Because the whole operation happens in a single step, no interrupt (or other processor) can interfere.

Atomic bit operations are very fast, since they perform the operation using a single machine instruction without disabling interrupts whenever the underlying platform can do that. The functions are architecture dependent and are declared in *<asm/bitops.h>*. They are guaranteed to be atomic even on SMP computers and are useful to keep coherence across processors.

Unfortunately, data typing in these functions is architecture dependent as well. The `nr` argumen t (describing which bit to manipulate) is usually defined as `int` but is `unsigned long` for a few archi tectures. The addres s to be modified is usually a pointer to

`unsigned long`, but a few architectures use `void *` instead.

The available bit operations are:

```
void set_bit(nr, void *addr);
```

Sets bit number `nr` in the data item pointed to by `addr`.

```
void clear_bit(nr, void *addr);
```

Clears the specified bit in the `unsigned long` datum that lives at `addr`. Its semantics are otherwise the same as *set_bit*.

```
void change_bit(nr, void *addr);
```

Toggles the bit.

```
test_bit(nr, void *addr);
```

This function is the only bit operation that doesn't need to be atomic; it simply returns the current valu e of the b it.

```
int test_and_set_bit(nr, void *addr);
int test_and_clear_bit(nr, void *addr);
int test_and_change_bit(nr, void *addr);
```

Behave atomically like those listed previously, except that they also return the previous value of the bit.

When these functions are used to access and modify a shared flag, you don't have to do anything except call them; they perform their operations in an atomic manner. Using bit operations to manage a lock va riable that controls access to a shared variable, on the other hand, is a little more complicated and deserves an example. Most modern code does not use bit operations in this way, but code like the f ollowing s till exist s in the kernel.

A code segment that needs to access a shared data item tries to atomically acquire a lock using either *test_and_set_bit* or *test_and_clear_bit*. The usual implementation is shown here; it assumes that the lock lives at bit `nr` of address `addr`. It also assumes that the bit is `0` when the l ock is fre e or nonze ro when th e lock is busy.

```
/* try to set lock */
while (test_and_set_bit(nr, addr) != 0)
    wait_for_a_while(  );

/* do your work */

/* release lock, and check... */
if (test_and_clear_bit(nr, addr) =  = 0)
    something_went_wrong(  ); /* already released: error */
```

If you read through the kernel source, you find code that works like this example. It is, however, far better to use spinlocks in new code; spinlocks are well debugged, they handle issues like interrupts and kernel preemption, and others reading your code do not have to work to understand what you are doing.

seqlocks

The 2.6 kernel contains a couple of new mechanisms that are intended to provide fast, lockless access to a shared resource. Seqlocks work in situations where the resource to be protected is small, simple, and frequently accessed, and where write access is rare but must be fast. Essentially, they work by allowing readers free access to the resource but requiring those readers to check for collisions with writers and, when such a collision happens, retry their access. Seqlocks generally cannot be used to protect data structures involving pointers, because the reader may be following a pointer that is invalid while the writer is changing the data structure.

Seqlocks are defined in *<linux/seqlock.h>*. There are the two usual methods for initializing a seqlock (which has type `seqlock_t`):

```
seqlock_t lock1 = SEQLOCK_UNLOCKED;

seqlock_t lock2;
seqlock_init(&lock2);
```

Read access works by obtain ing an (un signed) integer sequence value on entry into the critical section. On exit, that sequence value is compared with the current value; if there is a mismatch, the read access must be retried. As a result, reader code has a form like the following:

```
unsigned int seq;

do {
    seq = read_seqbegin(&the_lock);
    /* Do what you need to do */
} while read_seqretry(&the_lock, seq);
```

This sort of lock is usually used to protect some sort of simple computation that requires multiple, consistent values. If the test at the end of the computation shows that a concurrent wri te occurred, the results can be simply discarded and recomputed.

If your seqlock might be accessed from an interrupt handler, you should use the IRQ-safe versions instead:

```
unsigned int read_seqbegin_irqsave(seqlock_t *lock,
                                   unsigned long flags);
int read_seqretry_irqrestore(seqlock_t *lock, unsigned int seq,
                             unsigned long flags);
```

Writers must obtain an exclusive lock to enter the critical section protected by a seqlock. To do so, call:

```
void write_seqlock(seqlock_t *lock);
```

The write lock is implemented with a spinlock, so all the usual constraints apply. Make a call to:

```
void write_sequnlock(seqlock_t *lock);
```

to release the lock. Since spinlocks are used to control write access, all of the usual variants are available:

```
void write_seqlock_irqsave(seqlock_t *lock, unsigned long flags);
void write_seqlock_irq(seqlock_t *lock);
void write_seqlock_bh(seqlock_t *lock);

void write_sequnlock_irqrestore(seqlock_t *lock, unsigned long flags);
```

```
void write_sequnlock_irq(seqlock_t *lock);
void write_sequnlock_bh(seqlock_t *lock);
```

There is also a *write_tryseqlock* that returns nonzero if it was able to obtain the lock.

Read-Copy-Update

Read-copy-update (RCU) is an advanced mutual exclusion scheme that can yield high performance in the right conditions. Its use in drivers is rare but not unknown, so it is worth a quick overview here. Those who are interested in the full details of the RCU algorithm can find them in the white paper published by its creator (http://www.rdrop.com/users/paulmck/rclock/intro/rclock_intro.html).

RCU places a number of constraints on the sort of data structure that it can protect. It is optimized for situations where reads are common and writes are rare. The resources being protected should be accessed via pointers, and all references to those resources must be held only by atomic code. When the data structure needs to be changed, the writing thread makes a copy, changes the copy, then aims the relevant pointer at the new version — thus, the name of the algorithm. When the kernel is sure that no references to the old versio n remain, it can be freed.

As an example of real-world use of RCU, consid er the net work routi ng tables. Every outgoing packet requires a check of the routing tables to determine which interface should be used. The check is fast, and, once the kernel has found the target interface, it no longer needs the routing table entry. RCU allows route lookups to be performed without locking, with significant performance benefits. The Starmode radio IP driver in the kernel also uses RCU to keep track of its list of devices.

Code using RCU should include *<linux/rcupdate.h>*.

On the read side, code using an RCU-protected data structure should bracket its references with calls to *rcu_read_lock* and *rcu_read_unlock*. As a result, RCU code tends to look like:

```
struct my_stuff *stuff;

rcu_read_lock(  );
stuff = find_the_stuff(args...);
do_something_with(stuff);
rcu_read_unlock(  );
```

The *rcu_read_lock* call is fast; it disables kernel preemption but does not wait for anything. The code that executes while the read “lock” is held must be atomic. No reference to the protected resource may be used after the call to *rcu_read_unlock*.

Code that needs to change the protected structure has to carry out a few steps. The first part is easy; it allocates a new structure, copies data from the old one if need be, then replaces the pointer that is seen by the read code. At this point, for the purposes of the read side, the change is complete; any code entering the critical section sees the new version of the data.

All that remains is to free the old version. The problem, of course, is that code running on other processors may still have a reference to the older data, so it cannot be freed immediately. Instead, the write code must wait until it knows that no such reference can exist. Since all code holding references to this data structure must (by the rules) be atomic, we know that once every processor on the system has been scheduled at least once, all references must be gone. So that is what RCU does; it sets aside a callback that waits until all processors have scheduled; that callback is then run to perform the cleanup work.

Code that changes an RCU-protected data structure must get its cleanup callback by allocating a `struct rcu_head`, although it doesn’t need to initialize that structure in any way. Often, that structure is simply embedded within the larger resource that is protected by RCU. After the change to that resource is complete, a call should be made to:

```
void call_rcu(struct rcu_head *head, void (*func)(void *arg), void *arg);
```

The given `func` is called when it is safe to free the resource; it is passed to the same `arg` that was passed to *call_rcu*. Usually, the only thing `func` needs to do is to call *kfree*.

The full RCU interface is more complex than we have seen here; it includes, for example, utility functions for working with protected linked lists. See the relevant header files for the full story.

Quick Reference

This chapter has introduced a substantial set of symbols for the management of concurrency. The most important of these are summarized here:

`#include` <asm/semaphore.h>

> The include file that defines semaphores and the operations on them.

```
DECLARE_MUTEX(name);
DECLARE_MUTEX_LOCKED(name);
```

> Two macros for declaring and initializing a semaphore used in mutual exclusion mode.

```
void  init_MUTEX (struct se maphore *s em);
void init_MUTEX_LOCKED(struct semaphore *sem);
```

> These two functions ca n be used to initialize a semaphore at runtime.

```
void down(struct semaphore *sem);
int down_interruptible(struct semaphore *sem);
int down_trylock(struct semaphore *sem);
void up(struct semaphore *sem);
```

> Lock and unlock a semaphore. *down* puts the calling process into an uninterruptible sleep if need be; *down_interruptible*, instead, can be interrupted by a signal. *down_trylock* does not sleep; instead, it returns immediately if the semaphore is unavailable. Code that locks a semaphore must eventually unlock it with *up*.

```
struct rw_semaphore;
init_rwsem(struct rw_semaphore *sem);
```

> The reader/writer version of semaphores and the function that initializes it.

```
void down_read(struct rw_semaphore *sem);
```

```
int down_read_trylock(struct rw_semaphore *sem);
void up_read(struct rw_semaphore *sem);
```

Functions for obtaining and releasing read access to a reader/writer semaphore.

```
void down_write(struct rw_semaphore *sem)
int down_write_trylock(struct rw_semaphore *sem)
void up_write(struct rw_semaphore *sem)
void downgrade_write(struct rw_semaphore *sem)
```

Functions for managing write access to a reader/writer semaphore.

```
#include <linux/completion.h>
DECLARE_COMPLETION(name);
init_completion(struct completion *c);
INIT_COMPLETION(struct completion c);
```

The include file describing the Linux completion mechanism, and the normal methods for initializing completions. `INIT_COMPLETION` should be used only to reinitialize a completion that has been previously used.

```
void wait_for_completion(struct completion *c);
```

Wait for a completion event to be signalled.

```
void complete(struct completion *c);
void complete_all(struct completion *c);
```

Signal a completion event. *complete* wakes, at most, one waiting thread, while *complete_all* wakes all waiters.

```
void complete_and_exit(struct completion *c, long retval);
```

Signals a completion event by calling *complete* and calls *exit* for the current thread.

```
#include <linux/spinlock.h>
spinlock_t lock = SPIN_LOCK_UNLOCKED;
spin_lock_init(spinlock_t *lock);
```

The include file defining the spinlock interface and the two ways of initializing locks.

```
void spin_lock(spinlock_t *lock);
void spin_lock_irqsave(spinlock_t *lock, unsigned long flags);
void spin_lock_irq(spinlock_t *lock);
void spin_lock_bh(spinlock_t *lock);
```

The various ways of locking a spinlock and, possibly, disabling interrupts.

```
int spin_trylock(spinlock_t *lock);
int spin_trylock_bh(spinlock_t *lock);
```

Nonspinning versions of the above functions; these return `0` in case of failure to obtain the lock, nonzero otherwise.

```
void spin_unlock(spinlock_t *lock);
void spin_unlock_irqrestore(spinlock_t *lock, unsigned long flags);
void spin_unlock_irq(spinlock_t *lock);
void spin_unlock_bh(spinlock_t *lock);
```

The corresponding ways of releasing a spinlock.

```
rwlock_t lock = RW_LOCK_UNLOCKED
rwlock_init(rwlock_t *lock);
```

The two ways of initializing reader/writer locks.

```
void read_lock(rwlock_t *lock);
void read_lock_irqsave(rwlock_t *lock, unsigned long flags);
void read_lock_irq(rwlock_t *lock);
void read_lock_bh(rwlock_t *lock);
```

Functions for obtaining read access to a reader/writer lock.

```
void read_unlock(rwlock_t *lock);
void read_unlock_irqrestore(rwlock_t *lock, unsigned long flags);
void read_unlock_irq(rwlock_t *lock);
void read_unlock_bh(rwlock_t *lock);
```

Functions for releasing read access to a reader/writer spinlock.

```
void write_lock(rwlock_t *lock);
void write_lock_irqsave(rwlock_t *lock, unsigned long flags);
void write_lock_irq(rwlock_t *lock);
void write_lock_bh(rwlock_t *lock);
```

Functions for obtaining write access to a reader/writer lock.

```
void write_unlock(rwlock_t *lock);
void write_unlock_irqrestore(rwlock_t *lock, unsigned long flags);
void write_unlock_irq(rwlock_t *lock);
void write_unlock_bh(rwlock_t *lock);
```

Functions for releasing write access to a reader/writer spinlock.

```
#include <asm/atomic.h>
atomic_t v = ATOMIC_INIT(value);
void atomic_set(atomic_t *v, int i);
int atomic_read(atomic_t *v);
void atomic_add(int i, atomic_t *v);
void atomic_sub(int i, atomic_t *v);
void atomic_inc(atomic_t *v);
void atomic_dec(atomic_t *v);
int atomic_inc_and_test(atomic_t *v);
int atomic_dec_and_test(atomic_t *v);
int atomic_sub_and_test(int i, atomic_t *v);
int atomic_add_negative(int i, atomic_t *v);
int atomic_add_return(int i, atomic_t *v);
int atomic_sub_return(int i, atomic_t *v);
int atomic_inc_return(atomic_t *v);
int atomic_dec_return(atomic_t *v);
```

Atomically access integer variables. The `atomic_t` variables must be accessed only through these functions.

```
#include <asm/bitops.h>
void set_bit(nr, void *addr);
void clear_bit(nr, void *addr);
void change_bit(nr, void *addr);
```

```
test_bit(nr, void *addr);
int test_and_set_bit(nr, void *addr);
int test_and_clear_bit(nr, void *addr);
int test_and_change_bit(nr, void *addr);
```

Atomically access bit values; they can be used for flags or lock variables. Using these functions prevents any race condition related to concurrent access to the bit.

```
#include <linux/seqlock.h>
seqlock_t lock = SEQLOCK_UNLOCKED;
seqlock_init(seqlock_t *lock);
```

The include file defining seqlocks and the two ways of initializing them.

```
unsigned int read_seqbegin(seqlock_t *lock);
unsigned int read_seqbegin_irqsave(seqlock_t *lock, unsigned long flags);
int read_seqretry(seqlock_t *lock, unsigned int seq);
int read_seqretry_irqrestore(seqlock_t *lock, unsigned int seq, unsigned long
flags);
```

Functions for obtaining read access to a seqlock-protected resources.

```
void write_seqlock(seqlock_t *lock);
void write_seqlock_irqsave(seqlock_t *lock, unsigned long flags);
void write_seqlock_irq(seqlock_t *lock);
void write_seqlock_bh(seqlock_t *lock);
int write_tryseqlock(seqlock_t *lock);
```

Functions for obtaining write access to a seqlock-protected resource.

```
void write_sequnlock(seqlock_t *lock);
void write_sequnlock_irqrestore(seqlock_t *lock, unsigned long flags);
void write_sequnlock_irq(seqlock_t *lock);
void write_sequnlock_bh(seqlock_t *lock);
```

Functions for releasing write access to a seqlock-protected resource.

```
#include <linux/rcupdate.h>
```

The include file required to use the read-copy-update (RCU) mechanism.

```
void rcu_read_lock;
void rcu_read_unlock;
```

Macros for obtaining atomic read access to a resource protected by RCU.

```
void call_rcu(struct rcu_head *head, void (*func)(void *arg), void *arg);
```

Arranges for a callback to run after all processors have been scheduled and an RCU-protected resource can be safely freed.

---

[1] For the purposes of this chapter, a “thread” of execution is any context that is running code. Each process is clearly a thread of execution, but so is an interrupt handler or other code running in response to an asynchronous kernel event.

[2] As of this writing, patches adding interruptible versions were in circulation but had not been merged into the mainline.

[3] This lock still exists in 2.6, though it covers very little of the kernel now. If you stumble across a *lock_kernel* call, you have found the big kernel lock. Do not even think about using it in any new code, however.

Chapter 7. Time, Delays, and Deferred Work

At this point, we know the basics of how to write a full-featured char module. Real-world drivers, however, need to do more than implement the operations that control a device; they have to deal with issues such as timing, memory management, hardware access, and more. Fortunately, the kernel exports a number of facilities to ease the task of the driver writer. In the next few chapters, we'll describe some of the kernel resources you can use. This chapter leads the way by describing how timing issues are addressed. Dealing with time involves the following tasks, in order of increasing complexity:

- Measuring time lapses and comparing times
- Knowing the current time
- Delaying operation for a specified amount of time
- Scheduling asynchronous functions to happen at a later time

Measuring Time Lapses

The kernel keeps track of the flow of time by means of timer interrupts. Interrupts are covered in detail in Chapter 10.

Timer interrupts are generated by the system's timing hardware at regular intervals; this interval is programmed at boot time by the kernel according to the value of `HZ`, which is an architecture-dependent value defined in *<linux/param.h>* or a subplatform file included by it. Default values in the distributed kernel source range from 50 to 1200 ticks per second on real hardware, down to 24 for software simulators. Most platforms run at 100 or 1000 interrupts per second; the popular x86 PC defaults to 1000, although it used to be 100 in previous versions (up to and including 2.4). As a general rule, even if you know the value of `HZ`, you should never count on that specific value when programming.

It is possible to change the value of `HZ` for those who want systems with a different clock interrupt frequency. If you change `HZ` in the header file, you need to recompile the kernel and all modules with the new value. You might want to raise `HZ` to get a more fine-grained resolutio n in your asynchrono us tasks, if you are willing to pay the overhead of the extra timer interrupts to achieve your goals. Actually, raising `HZ` to 1000 was p retty comm on with x8 6 industrial systems using Version 2.4 or 2 .2 of the kernel. With current versions, however, the best approach to the timer interrupt is to keep the default value for `HZ`, by virtue of our complete trust in the kernel developers, who have certainly chosen the best value. Besides, some internal calculations are currently implemented only for `HZ` in the range from 12 to 1535 (see *<linux/timex.h>* and RFC-1589).

Every time a timer interrupt occurs, the value of an internal kernel counter is incremented. The counter is initialized to `0` at system boot, so it represents the number of clock ticks since last boot. The counter is a 64-bit variable (even on 32-bit architectures) and is called `jiffies_64`. However, driver writers normally access the `jiffies` variable, an `unsigned long` that is the same as either `jiffies_64` or its least significant bits. Using `jiffies` is usually preferred because it is faster, and accesses to the 64-bit `jiffies_64` value are not necessarily atomic on all architectures.

In addition to the low-resolution kernel-managed jiffy mechanism, some CPU platforms feature a high-resolution counter that software can read. Although its actual use varies somewhat across platforms, it's sometimes a very powerful tool.

Using the jiffies Counter

The counter an d the utility functions to read it live in *<linux/jiffies.h>*, although you'll usually just include *<linux/sched.h>*, that automatically pulls *jiffies.h* in. Needless to say, both `jiffies` and `jiffies_64` must be considered read-only.

Whenever your code needs to remember the current value of `jiffies`, it can simply access the `unsigned long` variable, which is declared as volatile to tell the compiler not to optimize memory reads. You need to read the current counter whenever your code needs to calculate a future time stamp, as shown in the following example:

```
#include <linux/jiffies.h>
unsigned long j, stamp_1, stamp_half, stamp_n;

j = jiffies;                      /* read the current value */
stamp_1    = j + HZ;              /* 1 second in the future */
stamp_half = j + HZ/2;            /* half a second  */
stamp_n    = j + n * HZ / 1000;   /* n milliseconds */
```

This code has no problem with `jiffies` wrapping around, as long as different values are compared in the right way. Even though on 32-bit platforms the counter wraps around only once every 50 days when `HZ` is 1000, your code should be prepared to face that event. To compare your cached value (like `stamp_1` above) and the current value, you should use one of the following macros:

```
#include <linux/jiffies.h>
int time_after(unsigned long a, unsigned long b);
int time_before(unsigned long a, unsigned long b);
int time_after_eq(unsigned long a, unsigned long b);
int time_before_eq(unsigned long a, unsigned long b);
```

The first evaluates true when *a*, as a snapshot of `jiffies`, represents a time after *b*, the second evaluates true when time *a* is before time *b*, and the last two compare for "after or equal" and "before or equal." The code works by converting the values to signed long, subtracting them, and comparing the result. If you need to know the difference between two instances of `jiffies` in a safe way, you can use the same trick: `diff = (long)t2 - (long)t1;`.

You can convert a jiffies difference to milliseconds trivially through:

```
msec = diff * 1000 / HZ;
```

Sometimes, however, you need to exchange time representations with user space programs that tend to represent time values with `struct timeval` and `struct timespec`. The two structures represent a precise time quantity with two numbers: seconds and microseconds are used in the older and popular `struct timeval`, and seconds and nanoseconds are used in the newer `struct timespec`. The kernel exports four helper functions to convert time values expressed as jiffies to and from those structures:

```
#include <linux/time.h>

unsigned long timespec_to_jiffies(struct timespec *value);
```

```
void jiffies_to_timespec(unsigned long jiffies, struct timespec *value);
unsigned long timeval_to_jiffies(struct timeval *value);
void jiffies_to_timeval(unsigned long jiffies, struct timeval *value);
```

Accessing the 64-bit jiffy count is not as straightforward as accessing `jiffies`. While on 64-bit computer architectures the two variables are actually one, access to the value is not atomic for 32-bit processors. This means you might read the wrong value if both halves of the variable get updated while you are reading them. It's extremely unlikely you'll ever need to read the 64-bit counter, but in case you do, you'll be glad to know that the kernel exports a specific helper function that does the proper locking for you:

```
#include <linux/jiffies.h>
u64 get_jiffies_64(void);
```

In the above prototype, the `u64` type is used. This is one of the types defined by <*linux/types.h*> and represents an unsigned 64-bit type.

If you're wondering how 32-bit platforms update both the 32-bit and 64-bit counters at the same time, read the linker script for your platform (look for a file whose name matches *vmlinux***.lds**). There, the `jiffies` symbol is defined to access the least significant word of the 64-bit value, according to whether the platform is little-endian or big-endian. Actually, the same trick is used for 64-bit platforms, so that the `unsigned long` and `u64` variables are accessed at the same address.

Finally, note that the actual clock frequency is almost completely hidden from user space. The macro `HZ` always expands to 100 when user-space programs include *param.h*, and every counter reported to user space is converted accordingly. This applies to *clock(3)*, *times(2)*, and any related function. The only evidence available to users of the `HZ` value is how fast timer interrupts happen, as shown in */proc/interrupts*. For example, you can obtain `HZ` by dividing this count by the system uptime reported in */proc/uptime*.

Processor-Specific Registers

If you need to measure very short time intervals or you need extremely high precision in your figures, you can resort to platform-dependent resources, a choice of precision over portability.

In modern processors, the pressing demand for empirical performance figures is thwarted by the intrinsic unpredictability of instruction timing in most CPU designs due to cache memories, instruction scheduling, and branch prediction. As a response, CPU manufacturers introduced a way to count clock cycles as an easy and reliable way to measure time lapses. Therefore, most modern processors include a counter register that is steadily incremented once at each clock cycle. Nowadays, this clock counter is the only reliable way to carry out high-resolution timekeeping tasks.

The details differ from platform to platform: the register may or may not be readable from user space, it may or may not be writable, and it may be 64 or 32 bits wide. In the last case, you must be prepared to handle overflows just like we did with the jiffy counter. The register may even not exist for your platform, or it can be implemented in an external device by the hardware designer, if the CPU lacks the feature and you are dealing with a special-purpose computer.

Whether or not the register can be zeroed, we strongly discourage re setting it, even when hardware permits. You might not, after all, be the only user of the counter at any given time; on some platforms supporting SMP, for examp le, the ke rnel depends on such a counter to be synchronized across processors. Since you can always measure differences between values, as long as that difference doesn't exceed the overflow time, you can get the work done without claiming exclusive ownership of the register by modifying its current value.

The most renowned counter register is the TSC (timestamp counter), introduced in x86 processors with the Pentium and present in all CPU designs ever since — including the x86_64 platform. It is a 64-bit register that counts CPU clock cycles; it can be read from both kernel space and user space.

After including <*asm/msr.h*> (an x86-specific header whose name stands for "machine-specific registers"), you can use one of these macros:

```
rdtsc(low32,high32);
rdtscl(low32);
rdtscll(var64);
```

The first macro atomically reads the 64-bit value into two 32-bit variables; the next one ("read low half") reads the low half of the register into a 32-bit variable, discarding the high half; the last reads the 64-bit value into a `long long` variable, hence, the name. All of these macros store values into their arguments.

Reading the low half of the counter is enough for most common uses of the TSC. A 1-GHz CPU overflows it only once every 4.2 seconds, so you won't need to deal with multiregister variables if the time lapse you are benchmarking reliably takes less time. However, as CPU frequencies rise over time and as timing requirements increase, you'll most likely need to read the 64-bit counter more often in the future.

As an example using only the low half of the register, the following lines measure the execution of the instruction itself:

```
unsigned long ini, end;
rdtscl(ini); rdtscl(end);
printk("time lapse: %li\n", end - ini);
```

Some of the other platforms offer similar functionality, and kernel headers offer an architecture-independent function that you can use instead of *rdtsc*. It is called *get_cycles*, defined in <*asm/timex.h*> (included by <*linux/timex.h*>). Its prototype is:

```
 #include <linux/timex.h>
 cycles_t get_cycles(void);
```

This function is defined for every platform, and it always returns `0` on the platforms that have no cycle-counter register. The `cycles_t` type is an appropriate unsigned type to hold the value read.

Despite the availability of an architecture-independent function, we'd like to take the opportunity to show an example of inline assembly code. To this aim, we implement a *rdtscl* function for MIPS processors that works in the same way as the x86 one.

We base the example on MIPS because most MIPS processors feature a 32-bit counter as register 9 of their internal "coprocessor 0." To access the register, readable only from kernel space, you can define the following macro that executes a "move from coprocessor 0" assemb ly instruc tion:[1]

```
#define rdtscl(dest) \
```

```
_ _asm_ _ _ _volatile_ _("mfc0 %0,$9; nop" : "=r" (dest))
```

With this macro in place, the MIPS processor can execute the same code shown earlier for the x86.

With *gcc* inline assembly, the allocation of general-purpose registers is left to the compiler. The macro just shown uses `%0` as a placeholder for "argument 0," which is later specified as "any register (`r`) used as output (`=`)." The macro also state s that the output register must correspond to the C expression `dest`. The syntax for inline assembly is very powerful but somewhat complex, especially for architectures that have constraints on what each register can do (namely, the x86 family). The syntax is described in the *gcc* documentation, usually available in the *info* documentation tree.

The short C-code fragment shown in this section has been run on a K7-class x86 processor and a MIPS VR4181 (using the macro just described). The former repo rted a time lapse of 11 clock ticks and the latter just 2 clock ticks. The small figure was expected, since RISC processors usually execute one instruction per clock cycle.

There is one other thing worth knowing about timestamp counters: they are not necessarily synchronized across processors in an SMP system. To be sure of getting a coherent value, you should disable preemption for code that is querying the counter.

Knowing the Current Time

Kernel code can always retrieve a representation of the current time by looking at the value of `jiffies`. Usually, the fact that the value represents only the time since the last boot is not relevant to the driver, because its life is limited to the system uptime. As shown, drivers can use the current value of `jiffies` to calculate time intervals across events (for example, to tell double-clicks from single-clicks in input device drivers or calculate timeouts). In short, looking at `jiffies` is almost always sufficient when you need to measure time intervals. If you need very precise measurements for short time lapses, processor-specific registers come to the rescue (although they bring in serious portability issues).

It's quite unlikely that a driver will ever need to know the wall-clock time, expressed in months, days, and hours; the information is usually needed only by user programs such as *cron* and *syslogd*. Dealing with real-world time is usually best left to user space, where the C library offers better support ; besides, such code is often too policy-related to belong in the kernel. There *is* a kernel function that turns a wall-clock time into a `jiffies` value, however:

```
    #include <linux/time.h>
unsigned long mktime (unsigned int year, unsigned int mon,
                      unsigned int day, unsigned int hour,
                      unsigned int min, unsigned int sec);
```

To repeat: dealing directly with wall-clock time in a driver is often a sign that policy is being implemented and should therefore be questioned.

While you won't have to deal with human-readable representations of the time, sometimes you need to deal with absolute timestamp even in kernel space. To this aim, *<linux/time.h>* exports the *do_gettimeofday* function. When called, it fills a `struct timeval` pointer — the same one used in the *gettimeofday* system call — with the familiar seconds and microseconds values. The prototype for *do_gettimeofday* is:

```
 #include <linux/time.h>
 void do_gettimeofday(struct timeval *tv);
```

The source states that *do_gettimeofday* has "near microsecond resolution," because it asks the timing hardware what fraction of the current jiffy has already elapsed. The precision varies from one architecture to another, however, since it depends on the actual hardware mechanisms in use. For example, some *m68knommu* processors, Sun3 systems, and other *m68k* systems cannot offer more than jiffy resolution. Pentium systems, on the other hand, offer very fast and precise subtick measures by reading the timestamp counter described earlier in this chapter.

The c urrent tim e is also available (though with jiffy granularity) from the `xtime` variable, a `struct timespec` value. Direct use of this variable is discouraged because it is difficult to atomically access both the fields. Therefore, the kernel offers the utility function *current_kernel_time*:

```
#include <linux/time.h>
struct timespec current_kernel_time(void);
```

Code for retrieving the current time in the various ways it is available within the *jit* ("just in time") module in the source files provided on O'Reilly's FTP site. *jit* crea tes a file called */proc/c urrentime*, which returns the following items in ASCII when read:

- The current `jiffies` and `jiffies_64` values as hex numbers
- The current time as returned by *do_gettimeofday*
- The `timespec` returned by *current_kernel_time*

We chose to use a dynamic */proc* file to keep the boilerplate code to a minimum — it's not worth creating a whole device just to return a little textual information.

The file returns text lines continuously as long as the module is loaded; each *read* system call collects and returns one set of data, organized in two lines for better readability. Whenever you read multiple data sets in less than a timer tick, you'll see the difference between *do_gettimeofday*, which queries the hardware, and the other values that are updated only when the timer ticks.

```
 phon% head -8 /proc/currentime
0x00bdbc1f 0x0000000100bdbc1f 1062370899.630126
                              1062370899.629161488
0x00bdbc1f 0x0000000100bdbc1f 1062370899.630150
                              1062370899.629161488
0x00bdbc20 0x0000000100bdbc20 1062370899.630208
                              1062370899.630161336
0x00bdbc20 0x0000000100bdbc20 1062370899.630233
                              1062370899.630161336
```

In the screenshot above, there are two interesting things to note. First, the *current_kernel_time* value, though expressed in nanoseconds, has only clock-tick granularity; *do_gettimeofday* consistently reports a later time but not later than the next timer

tick. Second, the 64-bit jiffies counter has the least-significant bit of the upper 32-bit word set. This happens because the default value for `INITIAL_JIFFIES`, used at boot time to initialize the counter, forces a low-word overflow a few minutes after boot time to help detect problems related to that very overflow. This initial bias in the counter has no effect, because `jiffies` is unrelated to wall-clock time. In */proc/uptime*, where the kernel extracts the uptime from the counter, the initial bias is removed before conversion.

Delaying Execution

Device drivers often need to delay the execution of a particular piece of code for a period of time, usually to allow the hardware to accomplish some task. In this section we cover a number of different techniques for achieving delays. The circumstances of each situation determine which technique is best to use; we go over them all, and point out the advantages and disadvantages of each.

One important thing to consider is how the delay you need compares with the clock tick, considering the range of `HZ` across the various platforms. Delays that are reliably longer than the clock tick, and don't suffer from its coarse granularity, can make use of the system clock. Very short delays typically must be implemented with software loops. In between these two cases lies a gray area. In this chapter, we use the phrase "long" delay to refer to a multiple-jiffy delay, which can be as low as a few milliseconds on some platforms, but is still long as seen by the CPU and the kernel.

The following sections talk about the different delays by taking a somewhat long path from various intuitive but inappropriate solutions to the right solution. We chose this path because it all ows a more in-depth discussion of kernel issues related to timing. If you are eager to find the right code, just skim through the section.

Long Delays

Occa sionally a driver ne eds to delay execution for relatively long periods — more than one clock tick. There are a few ways of accomplishing this sort of delay; we start with the simplest technique, then proceed to the more advanced techniques.

Busy waiting

If you want to delay execution by a multiple of the clock tick, allowing some slack in the value, the easiest (though not recommended) implementation is a loop that monitors the jiffy counter. The *busy-waiting* implementation usually looks like the following code, where `j1` is the value of `jiffies` at the expiration of the delay:

```
while (time_before(jiffies, j1))
    cpu_relax(  );
```

The call to *cpu_relax* invokes an architecture-specific way of saying that you're not doing much with the processor at the moment. On many systems it does nothing at all; on symmetric multithreaded ("hyperthreaded") systems, it may yield the core to the other thread. In any case, this approach should definitely be avoided whenever po ssible. We show it here because on occasion you might want to run this code to better understand the internals of other code.

So let's look at how this code works. The loop is guaranteed to work because `jiffies` is declared as `volatile` by the kernel headers and, therefore, is fetch ed from memory any time some C code accesses it. Although technically correct (in that it works as designed), this busy loop severely degrades system performance. If you didn't configure your kernel for preemptive operation, the loop completely locks the processor for the duration of the delay; the scheduler never preempts a process that is r unning in kernel space, and the computer looks completely dead until time `j1` is reached. The problem is less serious if you are running a preemptive kernel, because, unless the code is holding a lock, some of the processor's time can be recovered for other uses. Busy waits are still expensive on preemptive systems, however.

Still worse, if interrupts happen to be disabled when you enter the loop, `jiffies` won't be updated, and the `while` condition remains true forever. Running a preemptive kernel won't help either, and you'll be forced to hit the big red button.

This implementation of delaying code is available, like the following ones, in the *jit* module. The */proc/jit** files created by the module delay a whole secon d each tim e you read a line of text, and lines are guaranteed to be 20 bytes each. If you want to test the busy-wait code, you can read */proc/jitbusy*, which busy-loops for one second for each line it returns.

Warning

Be sure to read, at most, one line (or a few lines) at a time from */proc/jitbusy*. The simplified kernel mechanism to register */proc* files invokes the *read* method over and over to fill the data buffer the user requested. Therefore, a command such as *cat /proc/jitbusy*, if it reads 4 KB at a time, freezes the computer for 205 seconds.

The suggested command to read */proc/jitbusy* is *dd bs=20 < /proc/jitbusy*, optionally specifying the number of blocks as well. Each 20-byte line returned by the file represents the value the jiffy counter had before and after the delay. This is a sample run on an otherwise unloaded computer:

```
phon% dd bs=20 count=5 < /proc/jitbusy
  1686518   1687518
   1687519   1688519
  1688520   1689520
  1689520   1690520
  1690521   1691521
```

All looks good: delays are exactly one second (1000 jiffies), and the next *read* system call starts immediately after the previous one is over. But let's see what happens on a system with a large number of CPU-intensive processes running (and nonpreemptive kernel):

```
phon% dd bs=20 count=5 < /proc/jitbusy
```

```
  1911226   1912226
  1913323   1914323
  1919529   1920529
  1925632   1926632
  1931835   1932835
```

Here, each *read* system call delays exactly one second, but the kernel can take more than 5 seconds before scheduling the *dd* process so it can issue the next system call. That's expected in a multitasking system; CPU time is shared between all running processes, and a CPU-intensive process has its dynamic priority reduced. (A discussion of scheduling policies is outside the scope of this book.)

The test under load shown above has been performed while running the *load50* sample program. This program forks a number of processes that do nothing, but do it in a CPU-intensive way. The program is part of the sample files accompanying this book, and forks 50 processes by default, although the number can be specified on the command line. In this chapter, and elsewhere in the book, the tests with a loaded system have been performed with *load50* running in an otherwise idle computer.

If you repeat the command while running a preemptible kernel, you'll find no noticeable difference on an otherwise idle CPU and the following behavior under load:

```
phon% dd bs=20 count=5 < /proc/jitbusy
 14940680  14942777
 14942778  14945430
 14945431  14948491
 14948492  14951960
 14951961  14955840
```

Here, there is no significant delay between the end of a system call and the beginning of the next one, but the individual delays are far longer than one second: up to 3.8 seconds in the example shown and increasing over time. These values demonstrate that the process has been interrupted during its delay, scheduling other processes. The gap between system calls is not the only scheduling option for this process, so no special delay can be seen there.

Yielding the processor

As we have seen, busy waiting imposes a heavy load on the system as a whole; we would like to find a better technique. The first change that comes to mind is to explicitly release the CPU when we're not interested in it. This is accomplished by calling the *schedule* function, declared in <*linux/sched.h*>:

```
while (time_before(jiffies, j1)) {
    schedule(  );
}
```

This loop can be tested by reading */proc/jitsched* as we read */proc/jitbusy* above. However, is still isn't optimal. The current process does nothing but release the CPU, but it remains in the run queue. If it is the only runnable process, it actually runs (it calls the scheduler, which selects the same process, which calls the scheduler, which . . .). In other words, the load of the machine (the average number of running processes) is at least one, and the idle task (process number `0`, also called *swapper* for historical reasons) never runs. Though this issue may seem irrelevant, running the idle task when the computer is idle relieves the processor's workload, decreasing its temperature and increasing its lifetime, as well as the duration of the batteries if the computer happens to be your laptop. Moreover, since the process is actually executing during the delay, it is accountable for all the time it consumes.

The behavior of */proc/jitsched* is actually similar to running */proc/jitbusy* under a preemptive kernel. This is a sample run, on an unloaded system:

```
phon% dd bs=20 count=5 < /proc/jitsched
  1760205   1761207
  1761209   1762211
  1762212   1763212
  1763213   1764213
  1764214   1765217
```

It's interesting to note that each *read* sometimes ends up waiting a few clock ticks more than requested. This problem gets worse and worse as the system gets busy, and the driver could end up waiting longer than expected. Once a process releases the processor with *schedule*, there are no guarantees that the process will get the processor back anytime soon. Therefore, calling *schedule* in this manner is not a safe solution to the driver's needs, in addition to being bad for the computing system as a whole. If you test *jitsched* while running *load50*, you can see that the delay associated to each line is extended by a few seconds, because other processes are using the CPU when the timeout expires.

Timeouts

The suboptimal delay loops shown up to now work by watching the jiffy counter without telling anyone. But the best way to implement a delay, as you may imagine, is usually to ask the kernel to do it for you. There are two ways of setting up jiffy-based timeouts, depending on whether your driver is waiting for other events or not.

If your driver uses a wait queue to wait for some other event, but you also want to be sure that it runs within a certain period of time, it can use *wait_event_timeout* or *wait_event_interruptible_timeout*:

```
#include <linux/wait.h>
long wait_event_timeout(wait_queue_head_t q, condition, long timeout);
long wait_event_interruptible_timeout(wait_queue_head_t q,
                      condition, long timeout);
```

These functions sleep on the given wait queue, but they return after the timeout (expressed in jiffies) expires. Thus, they implement a bounded sleep that does not go on forever. Note that the timeout value represents the number of jiffies to wait, not an absolute time value. The value is represented by a signed number, because it sometimes is the result of a subtraction, although the functions complain through a *printk* statement if the provided timeout is negative. If the timeout expires, the functions return `0` ; if the process is awakened by another event, it returns the remaining delay expressed in jiffies. The return value is never negative, even if the delay is greater than expected because of system load.

The */proc/jitqueue* file shows a delay based on *wait_event_interruptible_timeout*, although th e module h as no even t to wait for, and uses `0` as a condition:

```
wait_queue_head_t wait;
init_waitqueue_head (&wait);
wait_event_interruptible_timeout(wait, 0, delay);
```

The observed behaviour, when reading */proc/jitqueue*, is nearly optimal, even under load:

```
phon% dd bs=20 count=5 < /proc/jitqueue
  2027024   2028024
  2028025   2029025
  2029026   2030026
  2030027   2031027
  2031028   2032028
```

Since the reading process (*dd* above) is not in the run queue while waiting for the timeout, you see no difference in behavior whether the code is run in a preemptive kernel or not.

wait_event_timeout and *wait_event_interruptible_timeout* were designed with a hardware driver in mind, where execution could be resumed in either of two ways: either somebody calls *wake_up* on the wait queue, or the timeout expires. This doesn't apply to *jitqueue*, as nobody ever calls *wake_up* on the wait queue (after all, no other code even knows about it), so the process always wakes up when the timeout expires. To accommodate for this very situation, where you want to delay execution waiting for no specific event, the kernel offers the *schedule_timeout* function so you can avoid declaring and using a superfluous wait queue head:

```
#include <linux/sched.h>
signed long schedule_timeout(signed long timeout);
```

Here, `timeout` is the number of jiffies to delay. The return value is `0` unless the function returns before the given timeout has elapsed (in response to a signal). *schedule_timeout* requires that the caller first set the current process state, so a typical call looks like:

```
set_current_state(TASK_INTERRUPTIBLE);
schedule_timeout (delay);
```

The previous lines (from */proc/jitschedto*) cause the process to sleep until the given time has passed. Since *wait_event_interruptible_timeout* relies on *schedule_timeout* internally, we won't bother showing the numbers *jitschedto* returns, because they are the same as those of *jitqueue*. Once again, it is worth noting that an extra time interval could pass between the expiration of the timeout and when your process is actually scheduled to execute.

In the example just shown, the first line calls *set_current_state* to set things up so that the scheduler won't run the current process again until the timeout places it back in `TASK_RUNNING` state. To achieve an uninterruptible delay, use `TASK_UNINTERRUPTIBLE` instead. If you forget to change the state of the current process, a call to *schedule_timeout* behaves like a call to *schedule* (i.e., the *jitsched* behavior), setting up a timer that is not used.

If you want to play with the four *jit* files under different system situations or different kernels, or try other ways to delay execution, you may want to configure the amount of the delay when loading the module by setting the *delay* module parameter.

Short Delays

When a device driver needs to deal with latencies in its hardware, the delays involved are usually a few dozen microseconds at most. In this case, relying on the clock tick is definitely not the way to go.

The kernel functions *ndelay*, *udelay*, and *mdelay* serve well for short delays, delaying execution for the specified number of nanoseconds, microseconds, or milliseconds respectively.[2] Their prototypes are:

```
#include <linux/delay.h>
void ndelay(unsigned long nsecs);
void udelay(unsigned long usecs);
void mdelay(unsigned long msecs);
```

The actual implementations of the functions are in *<asm/delay.h>*, being architecture-specific, and sometimes build on an external function. Every architecture implements *udelay*, but the other functions may or may not be defined; if they are not, *<linux/delay.h>* offers a default version based on *udelay*. In all cases, the delay achieved is at least the requested value but could be more; actually, no platform currently achieves nanosecond precision, although several ones offer submicrosecond precision. Delaying mor e than the requested value is usually not a problem, as short delays in a driver are usually needed to wait for the hardw are, and the requirements are to wait for *at least* a given time lapse.

The implementation of *udelay* (and possibly *ndelay* too) uses a software loop based on the processor speed calculated at boot time, using the integer variable `loops_per_jiffy`. If you want to look at the actual code, however, be aware that the *x86* implementation is quite a complex one because of the different timing sources it uses, based on what CPU type is running the code.

To avoid integer overflows in loop calculations, *udelay* and *ndelay* impose an upper bound in the value passed to them. If your module fails to load and displays an unresolved s ymbol, *_ _bad_ude lay*, it means you called *udelay* with too large an argument. Note, however, that the compile-time check can be performed only on constant values and that not all platforms implement it. As a general rule, if you are trying to delay for thousands of nanoseconds, you should be using *udelay* rather than *ndelay*; similarly, millisecond-scale delays should be done with *mdelay* and not one of the finer-grained functions.

It's important to remember that the three delay functions are busy-waiting; other tasks can't be run during the time lapse. Thus, they replicate, though on a different scale, the behavior of *jitbusy*. Thus, these functions should only be used when ther e is no pr actical alternative.

There is another way of achieving millisecond (and longer) delays that does not involve busy waiting. The file *<linux/delay.h>* declares these functions:

```
void msleep(unsigned int millisecs);
unsigned long msleep_interruptible(unsigned int millisecs);
void ssleep(unsigned int seconds)
```

The first two functions puts the calling process to sleep for the given number of `millisecs`. A call to *msleep* is uninterruptible; you can be sure that the process sleeps for at least the given number of milliseconds. If your driver is sitting on a wait queue and you want a wakeup to break the sleep, use *msleep_interruptible*. The return value from *msleep_interruptible* is normally `0`; if, however, the process is awakened early, the return value is the number of milliseconds remaining in the originally requested sleep period. A call to *ssleep* puts the process into an uninterruptible sleep for the given number of seconds.

In general, if you can tolerate longer delays than requested, you should use *schedule_timeout*, *msleep*, or *ssleep*.

Kernel Timers

Whenever you need to schedule an action to happen later, without blocking the current process until that time arrives, kernel timers are the tool for you. These timers are used to schedule execution of a function at a particular time in the future, based on the clock tick, and can be used for a variety of tasks; for example, polling a device by checking its state at regular intervals when the hardware can't fire interrupts. Other typical uses of kernel timers are turning off the floppy motor or finishing another lengthy shut down operation. In such cases, delaying the return from *close* would impose an unnecessary (and surprising) cost on the application program. Finally, the kernel itself uses the timers in several situations, including the implementation of *schedule_timeout*.

A kernel timer is a data structure that instructs the kernel to execute a user-defined function with a user-defined argument at a user-defined time. The implementation resides in *<linux/timer.h>* and *kernel/timer.c* and is described in detail in the Section 7.4.2

The functions scheduled to run almost certainly do *not* run while the process that registered them is executing. They are, instead, run asyn chronously. Until now, everything we have done in our sample drivers has run in the context of a process executing syste m calls. When a timer runs, however, the process that scheduled it could be asleep, executing on a different processor, or quite possibly has exited altoge ther.

This asynchronous execution resembles what happens when a hardware interrupt happens (which is discussed in detail in Chapter 10). In fact, kernel timers are run as the result of a "software interrupt." When running in this sort of atomic context, your code is subject to a number of constraints. Timer functions must be atomic in all the ways we discussed in Chapter 5, but there are some additional issues brought about by the lack of a process context. We will introduce these constraints now; they will be seen again in several places in later chapters. Repetition is called for because the rules for atomic contexts must be followed assiduously, or the system will find itself in deep trouble.

A number of actions require the context of a process in order to be executed. When you are outside of process context (i.e., in interrupt context), you must observe the following rules:

- No access to user space is allowed. Because there is no process context, there is no path to the user space associated with any particular process.
- The `current` pointer is not meaningful in atomic mode and cannot be used since the relevant code has no connection with the process that has been interrupted.
- No sleeping or scheduling may be performed. Atomic code may not call *schedule* or a form of *wait_event*, nor may it call any other function that could sleep. For example, calling *kmalloc(..., GFP_KERNE L)* is against the rules. Semaphores also must not be used since they can sleep.

Kernel code can tell if it is running in interrupt context by calling the function *in_interrupt( )*, which takes no parameters and returns nonzero if the processor is currently running in interrupt context, either hardware interrupt or software interrupt.

A function related to *in_interrupt( )* is *in_atomic( )*. Its return value is nonzero whenever scheduling is not allowed; this includes hardware and software interrupt contexts as well as any time when a spinlock is held. In the latter case, `current` may be valid, but access to user space is forbidden, since it can cause scheduling to happen. Whenever you are using *in_interrupt( )*, you should really consider whether *in_atomic( )* is what you actually mean. Both functions are declared in *<asm/hardirq.h>*

One other important feature of kernel timers is that a task can reregister itself to run again at a later time. This is possible because each `timer_list` structure is unlinked from the list of active timers before being run and can, therefore, be immediately re-linked elsewhere. Although rescheduling the same task over and over might appear to be a pointless operation, it is sometimes useful. For example, it can be used to implement the polling of devices.

It's also worth knowing that in an SMP syst em, the ti mer function is executed by the same CPU that registered it, to achieve better cache locality whenever possible. Therefore, a timer that reregisters itself always runs on the same CPU.

An important feature of timers that should not be forgotten, though, is that they are a potential s ource of r ace conditions, even on uniprocessor systems. This is a direct result of their being asynchronous with other code. Therefore, any data structures accessed by the timer function should be protected from concurrent access, either by being atomic types (discussed in the section Atomic Variables) or by using spinlocks (discussed in Chapter 5).

The Timer API

The kernel provides drivers with a number of functions to declare, register, and remove kernel timers. The following excerpt shows the bas ic building blocks:

```
#include <linux/timer.h>
struct timer_list {
        /* ... */
        unsigned long expires;
        void (*function)(unsigned long);
        unsigned long data;
};

void init_timer(struct timer_list *timer);
struct timer_list TIMER_INITIALIZER(_function, _expires, _data);

void add_timer(struct timer_list * timer);
int del_timer(struct timer_list * timer);
```

The data structure includes more fields than the ones shown, but those three are the ones that are meant to be accessed from outside th e timer code iteslf. The `expires` field represents the `jiffies` value when the timer is expected to run; at that time, the function *function* is called with `data` as an argument. If you need to pass multiple items in the argument, you can bundle them as a single data structure and pass a pointer cast to `unsigned long`, a safe practice on all supported architectures and pretty common in memory management (as discussed in Chapter 15). The `expires` value is not a `jiffies_64` item because timers are not expected to expire very far in the future, and 64-bit operations are slow on 32-bit platforms.

The structure must be initialized before use. This step ensures that all the fields are properly set up, including the ones that are opaque to the caller. Initialization can be performed by calling *init_timer* or assigning `TIMER_INITIALIZER` to a static structure, according to your needs. After initialization, you can change the three public fields before calling *add_timer*. To disable a registered timer before it expires, call *del_timer*.

The *jit* module includes a sample file, */proc/jitimer* (for "just in timer"), that returns one header line and six data lines. The data lines represent the current environment where the code is running; the first one is generated by the *read* file operation and the others by a timer. The following output was recorded while compiling a kernel:

```
phon% cat /proc/jitimer
   time   delta  inirq    pid   cpu command
 33565837    0     0      1269   0   cat
 33565847   10     1      1271   0   sh
 33565857   10     1      1273   0   cpp0
 33565867   10     1      1273   0   cpp0
 33565877   10     1      1274   0   cc1
 33565887   10     1      1274   0   cc1
```

In this output, the `time` field is the value of `jiffies` when the code runs, `delta` is the change in `jiffies` since the previous line, `inirq` is the Boolean value returned by *in_interrupt*, `pid` and `command` refer to the current process, and `cpu` is the number of the CPU being used (always `0` on uniprocessor systems).

If you read */proc/jitimer* while the system is unloaded, you'll find that the context of the timer is process `0`, the idle task, which is called "swapper" mainly for historical reasons.

The timer used to generate */proc/jitimer* data is run every 10 jiffies by default, but you can change the value by setting the `tdelay` (timer delay) parameter when loading the module.

The following code excerpt shows the part of *jit* related to the *jitimer* timer. When a process attempts to read our file, we set up the timer as follows:

```
unsigned long j = jiffies;

/* fill the data for our timer function */
data->prevjiffies = j;
data->buf = buf2;
data->loops = JIT_ASYNC_LOOPS;

/* register the timer */
data->timer.data = (unsigned long)data;
data->timer.function = jit_timer_fn;
data->timer.expires = j + tdelay; /* parameter */
add_timer(&data->timer);

/* wait for the buffer to fill */
wait_event_interruptible(data->wait, !data->loops);
```

The actual timer function looks like this:

```
void jit_timer_fn(unsigned long arg)
{
    struct jit_data *data = (struct jit_data *)arg;
    unsigned long j = jiffies;
    data->buf += sprintf(data->buf, "%9li  %3li     %i    %6i   %i   %s\n",
                 j, j - data->prevjiffies, in_interrupt(  ) ? 1 : 0,
                 current->pid, smp_processor_id(  ), current->comm);

    if (--data->loops) {
        data->timer.expires += tdelay;
        data->prevjiffies = j;
        add_timer(&data->timer);
    } else {
        wake_up_interruptible(&data->wait);
    }
}
```

The timer API includes a few more functions than the ones introduced above. The following set completes the list of kernel offerings:

```
int mod_timer(struct timer_list *timer, unsigned long expires);
```

> Updates the expiration time of a timer, a common task for which a timeout timer is used (again, the motor-off floppy timer is a typical example). *mod_timer* can be called on inactive timers as well, where you normally use *add_timer*.

```
int del_timer_sync(struct timer_list *timer);
```

> Works like *del_timer*, but also guarantees that when it returns, the timer function is not running on any CPU. *del_timer_sync* is used to avoid race conditions on SMP systems and is the same as *del_timer* in UP kernels. This function should be preferred over *del_timer* in most situations. This function can sleep if it is called from a nonatomic context but busy waits in other situations. Be very careful about calling *del_timer_sync* while holding locks; if the timer function attempts to obtain the same lock, the system can deadlock. If the timer function reregisters itself, the caller must first ensure that this reregistration will not happen; this is usually accomplished by setting a "shutting down" flag, which is checked by the timer function.

```
int timer_pending(const struct timer_list * timer);
```

> Returns true or false to indicate whether the timer is currently scheduled to run by reading one of the opaque fields of the structure.

The Implementation of Kernel Timers

Although you won't need to know how kernel timers are implemented in order to use them, the implementation is interesting, and a look at its internals is worthwhile.

The implementation of the timers has been designed to mee t the foll owing requirements and assumptions:

- Timer management must be as lightweight as possible.
- The design should scale well as the number of active timers increases.
- Most timers expire within a few seconds or minutes at most, while timers with long delays are pretty rare.

- A timer should run on the same CPU that registered it.

The solution devised by kernel developers is based on a per-CPU data structure. The *timer_list* structure includes a pointer to that data structure in its `base` field. If `base` is `NULL`, the timer is not scheduled to run; otherwise, the pointer tells which data structure (and, therefore, which CPU) runs it. Per-CPU data items are described in Section 8.5 in Section 7.1.1.

Whenever kernel code registers a timer (via *add_timer* or *mo d_timer*), the operation is eventually performed by *internal_add_timer* (in `kernel/timer.c`) which, in turn, adds the new timer to a double-linked list of timers within a "cascading table" associated to the current CPU.

The cascading table works like this: if the timer expires in the next to 255 jiffies, it is added to one of the 256 lists devoted to short-range timers using the least significant bits of the `expires` field. If it expires farther in the future (but before 16,384 jiffies), it is added to one of 64 lists based on bits 9-14 of the `expires` field. For timers expiring even farther, the same trick is used for bits 15-20, 21-26, and 27-31. Timers with an expire field pointing still farther in the future (something that can happen only on 64-bit platforms) are hashed with a delay value of `0xffffffff`, and timers with `expires` in the past are scheduled to run at the next timer tick. (A timer that is already expired may sometimes be registered in high-load situations, especially if you run a preemptible kernel.)

When _ _*run_timers* is fired, it executes all pending timers for the current timer tick. If `jiffies` is currently a multiple of 256, the function also rehashes one of the next-level lists of timers into the 256 short-term lists, possibly cascading one or more of the other levels as well, according to the bit representation of `jiffies`.

This approach, while exceedingly complex at first sight, performs very well both with few timers and with a large number of them. The time required to manage each active timer is independent of the number of timers already registered and is limited to a few logic operations on the binary representation of its `expires` field. The only cost associated with this implementation is the memory for the 512 list heads (256 short-term lists and 4 groups of 64 more lists) — i.e., 4 KB of storage.

The function _ _*run_timers*, as shown by */proc/jitimer*, is run in atomic context. In addition to the limitations we already described, this brings in an interesting feature: the timer expires at just the right time, even if you are not running a preemptible kernel, and the CPU is busy in kernel space. You can see what happens when you read */proc/jitbusy* in the background and */proc/jitimer* in the foreground. Although the system appears to be locked solid by the busy-waiting system call, the kernel timers still work fine.

Keep in mind, however, that a kernel timer is far from perfect, as it suffers from jitter and other artifacts induced by hardware interrupts, as well as other timers and other asynchronous tasks. While a timer associated with simple digital I/O can be enough for simple tasks like running a stepper motor or other amateur electronics, it is usually not suitable for production systems in industrial environments. For such tasks, you'll most likely need to resort to a real-time kernel extension.

Tasklets

Another kernel facility related to timing issues is the *tasklet* mechanism. It is mostly used in interrupt management (we'll see it again in Chapter 10.)

Tasklets resemble kernel timers in some ways. They are always run at interrupt time, they always run on the same CPU that schedules them, and they receive an `unsigned long` argument. Unlike kernel timers, however, you can't ask to execute the function at a specific time. By scheduling a tasklet, you simply ask for it to be executed at a later time chosen by the kernel. This behavior is especially useful with interrupt handlers, where the hardware interrupt must be managed as quickly as possible, but most of the data management can be safely delayed to a later time. Actually, a tasklet, just like a kernel timer, is executed (in atomic mode) in the context of a "soft interrupt," a kernel mechanism that executes asynchronous tasks with hardware interrupts enabled.

A tasklet exists as a data structure that must be initialized before use. Initialization can be performed by calling a specific function or by declaring the structure us ing certai n macros:

```
    #include <linux/interrupt.h>

struct tasklet_struct {
      /* ...  */
      void (*func)(unsigned long);
      unsigned long data;
};

void tasklet_init(struc t tasklet_ struct *t,
  ;  &n bsp;   ;void (*func)(unsigned long), unsigned long data);
DECLARE_TASKLET(name, func, data);
DECLARE_TASKLET_DISABLED(name, func, data);
```

Tasklets offer a number of interesting features:

- A tasklet can be disabled and re-enabled later; it won't be executed until it is enabled as many times as it has been disabled.
- Just like timers, a tasklet can reregister itself.
- A tasklet can be scheduled to execute at normal priority or high priority. The latter group is always executed first.
- Tasklets may be run immediately if the system is not under heavy load but never later than the next timer tick.
- A tasklets can be concurrent with other tasklets but is strictly serialized with respect to itself — the same tasklet never runs simultaneously on more than one processor. Also, as already noted, a tasklet always runs on the same CPU that schedules it.

The *jit* module includes two files, */proc/jitasklet* and */proc/jitasklethi*, that return the same data as */proc/jitimer*, introduced in Section 7.4 When you read one of the files, you get back a header and six data lines. The first data line describes the context of the calling process, and the other lines describe the context of successive runs of a tasklet procedure. This is a sample run while compiling a kernel:

```
phon% cat /proc/jitasklet
   time   delta  inirq    pid   cpu command
  6076139    0     0      4370   0   cat
  6076140    1     1      4368   0   cc1
  6076141    1     1      4368   0   cc1
  6076141    0     1         2   0   ksoftirqd/0
  6076141    0     1         2   0   ksoftirqd/0
  6076141    0     1         2   0   ksoftirqd/0
```

As confirmed by the above data, the tasklet is run at the next timer tick as long as the CPU is busy running a process, but it is run immediately when the CPU is otherwise idle. The kernel provides a set of *ksoftirqd* kernel threads, one per CPU, just to run "soft interrupt" handlers, such as the *tasklet_action* function. Thus, the final three runs of the tasklet take place in the context of the *ksoftirqd* kernel thread associated to CPU `0`. The *jitasklethi* implementation uses a high-priority tasklet, explained in an upcoming list of functions.

The actual code in *jit* that implements */proc/jitasklet* and */proc/jitasklethi* is almost identical to the code that implements */proc/jitimer*, but it uses the tasklet calls instead of the timer ones. The following list lays out in detail the kernel interface to tasklets after the tasklet structure has been initialized:

```
void tasklet_disable(struct tasklet_struct *t);
```

> This function disables the given tasklet. The tasklet may still be scheduled with *tasklet_schedule*, but its execution is deferred until the tasklet has been enabled again. If the tasklet is currently running, this function busy-waits until the tasklet exits; thus, after calling *tasklet_disable*, you can be sure that the tasklet is not running anywhere in the system.

```
void tasklet_disable_nosync(struct tasklet_struct *t);
```

> Disable the tasklet, but without waiting for any currently-running function to exit. When it returns, the tasklet is disabled and won't be scheduled in the future until re-enabled, but it may be still running on another CPU when the function returns.

```
void tasklet_enable(struct tasklet_struct *t);
```

> Enables a tasklet that had been previously disabled. If the tasklet has already been scheduled, it will run soon. A call to *tasklet_enable* must match each call to *tasklet_disable*, as the kernel keeps track of the "disable count" for each tasklet.

```
void tasklet_schedule(struct tasklet_struct *t);
```

> Schedule the tasklet for execution. If a tasklet is scheduled again before it has a chance to run, it runs only once. How ever, if it is scheduled *while* it runs, it runs again after it completes; this ensures that events occurring while other events are being processed receive due attention. This behavior also allows a tasklet to reschedule itself.

```
void tasklet_hi_schedule(struct tasklet_struct *t);
```

> Schedule the tasklet for execution with higher priority. When the soft interrupt handler runs, it deals with high-priority tasklets before other soft interrupt t asks, incl uding "normal" tasklets. Ideally, only tasks with low-latency requirements (such as filling the audio buffer) should use this function, to avoid the additional latencies introduced by other soft interrupt handlers. Actually, */proc/jitasklethi* shows no human-visible difference from */proc/jitasklet*.

```
void tasklet_kill(struct tasklet_struct *t);
```

> This function ensures that the tasklet is not scheduled to run again; it is usually called when a device is being closed or the module removed. If the tasklet is scheduled to run, the function waits until it has executed. If the tasklet reschedules itself, you must prevent it from rescheduling itself before calling *tasklet_kill*, as with *del_timer_sync*.

Tasklets are implemented in *kernel/softirq.c*. The two tasklet lists (normal and high-priority) are declared as per-CPU data structures, using the same CPU-affinity mechanism used by kernel timers. The data structure used in tasklet management is a simple linked list, because tasklets have none of the sorting requirements of kernel timers.

Workqueues

Workqueues are, superficially, similar to tasklets; they allow kernel code to request that a function be called at some future time. There are, however, some significant differences between the two, including:

- Tasklets run in software interrupt context with the result that all tasklet code must be atomic. Instead, workqueue functions run in the context of a special kernel process; as a result, they have more flexibility. In particular, workqueue functions can sleep.
- Tasklets always run on the processor from which they were or iginally s ubmitted. Workqueues work in the same way, by default.
- Kernel code can request that the execution of workqueue functions be delayed for an explicit interval.

The key difference between the two is that tasklets execute quickly, for a short period of time, and in atomic mode, while workqueue functions may have higher latency but need not be atomic. Each mechanism has situations where it is appropriate.

Wor kqueues have a type of `struct workqueue_struct`, which is defined in *<linux/workqueue.h>*. A workqueue must be explicitly created before use, using one of the following two functions:

```
struct workqueue_s truct *create_workqueue(const char *name);
struct workqueue_struct *create_singlethread_workqueue(const char *name);
```

Each workqueue has one or more dedicated process es ("kerne l threads"), which r un functions submitted to the queue. If you use *create_workqueue*, you get a workqueue that has a dedicated thread for each processor on the system. In many cases, all those threads are simply overkill; if a single worker thread will suffice, create the workqueue with *create_singlethread_workqueue* instead.

To submit a task to a workqueue, you need to fill in a `work_struct` structure. This can be done at compile time as follows:

```
DECLARE_WORK(name, void (*function)(void *), void *data);
```

Where `name` is the name of the structure to be declared, `function` is the function that is to be called from the workqueue, and `data` is a value to pass to that function. If you need to set up the `work_struct` structure at runtime, use the following two macros:

```
INIT_WORK(struct work_struct *work, void (*function)(void *), void *data);
PREPARE_WORK(struct work_struct *work, void (*function)(void *), void *data);
```

INIT_WORK does a more thorough job of initializing the structure; you should use it the first time that structure is set up. *PREPARE_WORK* does almost the same job, but it does not initialize the pointers used to link the `work_struct` structure into the workqueue. If there is any possibility that the structure may currently be submitted to a workqueue, and you need to change that structure, use *PREPARE_WORK* rather than *INIT_WORK*.

There are two functions for submitting work to a workqueue:

```
int queue_work(struct workqueue_struct *queue, struct work_struct *work);
int queue_delayed_work(struct workqueue_struct *queue,
                       struct work_struct *work, unsigned long delay);
```

Either one adds `work` to the given `queue`. If *queue_delayed_work* is used, however, the actual work is not performed until at least `delay` jiffies have passed. The return value from these functions is `0` if the work was successfully added to the queue; a nonzero result means that this `work_struct` structure was already waiting in the queue, and was not added a second time.

At some time in the future, the work function will be called with the given `data` value. The function will be running in the context of the worker thread, so it can sleep if need be — although you should be aware of how that sleep might affect any other tasks submitted to the same workqueue. What the function cannot do, however, is access user space. Since it is running inside a kernel thread, there simply is no user space to access.

Should you need to cancel a pending workqueue entry, you may call:

```
int cancel_delayed_work(struct work_struct *work);
```

The return value is nonzero if the entry was canceled before it began execution. The kernel guarantees that execution of the given entry will not be initiated after a call to *cancel_delayed_work*. If *cancel_delayed_work* returns `0`, however, the entry may have already been running on a different processor, and might still be running after a call to *cancel_delayed_work*. To be absolutely sure that the work function is not running anywhere in the system after *cancel_delayed_work* returns `0`, you must follow that call with a call to:

```
void flush_workqueue(struct workqueue_struct *queue);
```

After *flush_workqueue* returns, no work function submitted prior to the call is running anywhere in the system.

When you are done with a workqueue, you can get rid of it with:

```
void destroy_workqueue(struct workqueue_struct *queue);
```

The Shared Queue

A device driver, in many cases, does not need its own workqueue. If you only submit tasks to the queue occasionally, it may be more efficient to simply use the shared, default workqueue that is provided by the kernel. If you use this queue, however, you must be aware that you will be sharing it with others. Among other things, that means that you should not monopolize the queue for long periods of time (no long sleeps), and it may take longer for your tasks to get their turn in the processor.

The *jiq* (“just in queue”) module exports two files that demonstrate the use of the shared workqueue. They use a single `work_struct` structure, which is set up this way:

```
static struct work_struct jiq_work;

    /* this line is in jiq_init(  ) */
    INIT_WORK(&jiq_work, jiq_print_wq, &jiq_data);
```

When a process reads */proc/jiqwq*, the module initiates a series of trips through the shared workqueue with no delay. The function it uses is:

```
int schedule_work(struct work_struct *work);
```

Note that a different function is used when working with the shared queue; it requires only the `work_struct` structure for an argument. The actual code in *jiq* looks like this:

```
prepare_to_wait(&jiq_wait, &wait, TASK_INTERRUPTIBLE);
schedule_work(&jiq_work);
schedule(  );
finish_wait(&jiq_wait, &wait);
```

The actual work function prints out a line just like the *jit* module does, then, if need be, resubmits the `work_struct` structure into the workqueue. Here is *jiq_print_wq* in its entirety:

```
static void jiq_print_wq(void *ptr)
{
    struct clientdata *data = (struct clientdata *) ptr;

    if (! jiq_print (ptr))
        return;

    if (data->delay)
        schedule_delayed_work(&jiq_work, data->delay);
    els e
        schedule_work(&jiq_work);
}
```

If the user is reading the delayed device (*/proc/jiqwqdelay*), the work function resubmits itself in the delayed mode with *schedule_delayed_work*:

```
int schedule_delayed_work(struct work_struct *work, unsigned long delay);
```

If you look at the output from these two devices, it looks something like:

```
% cat /proc/jiqwq
    time  delta preempt   pid cpu command
  1113043     0       0     7   1 events/1
  1113043     0       0     7   1 events/1
  1113043     0       0     7   1 events/1
  1113043     0       0     7   1 events/1
  1113043     0       0     7   1 events/1
% cat /proc/jiqwqdelay
    time  delta preempt   pid cpu command
  1122066     1       0     6   0 events/0
  1122067     1       0     6   0 events/0
```

```
  1122068     1       0     6   0 events/0
  1122069     1       0     6   0 events/0
  1122070     1       0     6   0 events/0
```

When */proc/jiqwq* is read, there is no obvious delay between the printing of each line. When, instead, */proc/jiqwqdelay* is read, there is a delay of exactly one jiffy between each line. In either case, we see the same process name printed; it is the name of the kernel thread that implements the shared workqueue. The CPU number is printed after the slash; we never know which CPU will be running when the */proc* file is read, but the work function will always run on the same processor thereafter.

If you need to cancel a work entry submitted to the shared queue, you may use *cancel_delayed_work*, as described above. Flushing the shared workqueue requires a separate function, however:

```
void flush_scheduled_work(void);
```

Since you do not know who else might be using this queue, you never really know how long it might take for *flush_scheduled_work* to return.

Quick Reference

This chapter introduced the following symbols.

Timekeeping

```
#include <linux/param.h>
HZ
```

The `HZ` symbol specifies the number of clock ticks generated per second.

```
#include <linux/jiffies.h>
volatile unsigned long jiffies
u64 jiffies_64
```

The `jiffies_64` variable is incremented once for each clock tick; thus, it's incremented `HZ` times per second. Kernel code most often refers to `jiffies`, which is the same a s `jiffies_64` on 64-bit platforms and the least significant half of it on 32-bit platforms.

```
int time_after(unsigned long a, unsigned long b);
int time_before(unsigned long a, unsigned long b);
int time_after_eq(unsigned long a, unsigned long b);
int time_before_eq(unsigned long a, unsigned long b);
```

These Boolean expressions compare jiffies in a safe way, without problems in case of counter overflow and without the need to access `jiffies_64`.

```
  u64 get_jiffies_64(void);
```

Retrieves `jiffies_64` without race conditions.

```
#include <linux/time.h>
unsigned long timespec_to_jiffies(struct timespec *value);
   void jiffies_to_timespec(unsigned long jiffies, struct timespec *value);
unsigned long timeval_to_jiffies(struct timeval *value);
void jiffies_to_timeval(unsigned long jiffies, struct timeval *value);
```

Converts time representations between jiffies and other representations.

```
#include <asm/msr.h>
rdtsc(low32,high32);
rdtscl(low32);
rdtscll(var32);
```

x86-specific macros to read the timestamp counter. They read it as two 32-bit halves, read only the lower half, or read all of it into a `long long` variable.

```
#include <linux/timex.h>
cycles_t get_cycles(void);
```

Returns the timestamp counter in a platform-independent way. If the CPU offers no timestamp feature, `0` is returned.

```
#include <linux/time.h>
unsigned long mktime(year, mon, day, h, m, s);
```

Returns the number of seconds since the Epoch, based on the six `unsigned int` arguments.

```
void do_gettimeofday(struct timeval *tv);
```

Returns the current time, as seconds and microseconds since the Epoch, with the best resolution the hardware can offer. On most platforms the resolution is one microsecond or better, although some platforms offer only jiffies resolution.

```
struct timespec current_kernel_time(void);
```

Returns the current time with the resolution of one jiffy.

Delays

```
#include <linux/wait.h>
long wait_event_interruptible_timeout(wait_queue_head_t *q, condition, signed
long timeout);
```

Puts the current process to sleep on the wait queue, installing a timeout value expressed in jiffies. Use *schedule_timeout* (below) for noninterruptible sleeps.

```
#include <linux/sched.h>
signed long schedule_timeout(signed long timeout);
```

Calls the scheduler after ensuring that the current process is awakened at timeout expiration. The caller must invoke *set_current_state* first to put itself in an interruptible or noninterruptible sleep state.

```
#include <linux/delay.h>
void ndelay(unsigned long nsecs);
void udelay(unsigned long usecs);
void mdelay(unsigned long msecs);
```

Introduces delay s of an integer number of nanoseconds, microseconds, and milliseconds. The delay achieved is at least the requested value, but it can be more. The argument to each function must not exceed a platform-specific limit (usually a few thousands).

```
void msleep(unsigned int millisecs);
unsigned long msleep_interruptible(unsigned int millisecs);
void ssleep(unsigned int seconds);
```

Puts the process to sleep for the given number of milliseconds (or seconds, in the case of *ssleep*).

Kernel Timers

```
   #include <asm/hardirq.h>
int in_interrupt(void);
int in_atomic(void);
```

Returns a Boolean value telling whether the calling code is executing in interrupt context or atomic context. Interrupt context is outside of a process context, either during hardware or software interrupt processing. Atomic context is when you can't schedule either an interrupt context or a process's context with a spinlock held.

```
#include <linux/timer.h>
void init_timer(struct timer_list * timer);
struct timer_list TIMER_INITIALIZER(_function, _expires, _data);
```

This function and the static declaration of the timer structure are the two ways to initialize a `timer_list` data structure.

```
void add_timer(struct timer_list * timer);
```

Registers the timer structure to run on the current CPU.

```
int mod_timer(struct timer_list * timer, unsigned long expires);
```

Changes the expiration time of an already scheduled timer structure. It can also act as an alternative to *add_timer*.

```
int timer_pending(struct timer_list * timer);
```

Macro that returns a Boolean value stating whether the timer str ucture is already registered to run.

```
void del_timer(struct timer_list * timer);
void del_timer_sync(struct timer_list * timer);
```

Removes a timer from the list of active timers. The latter function ensures that the timer is not currently running on another CPU.

Tasklets

```
#include <linux/interrupt.h>
DECLARE_TASKLET(name, func, data);
DECLARE_TASKLET_DISABLED(name, func, data);
void tasklet_init(struct tasklet_struct *t, void (*func)(unsigned long),
unsigned long data);
```

The first two macros declare a tasklet structure, while the *tasklet_init* function initializes a tasklet structure that has been obtained by allocation or other means. The second `DECLARE` macro marks the tasklet as disabled.

```
void tasklet_disable(struct tasklet_struct *t);
void tasklet_disable_nosync(struct tasklet_struct *t);
void tasklet_enable(struct tasklet_struct *t);
```

Disables and reenables a tasklet. Each *disable* must be matched with an *enable* (you can disable the tasklet even if it's already disabled). The function *tasklet_disable* waits for the tasklet to terminate if it is running on another CPU. The *nosync* version doesn't take this extra step.

```
void tasklet_schedule(struct tasklet_struct *t);
void tasklet_hi_schedule(struct tasklet_struct *t);
```

Schedules a tasklet to run, either as a "normal" tasklet or a high-priority one. When soft interrupts are executed, high-priority tasklets are dealt with first, while normal tasklets run last.

```
void tasklet_kill(struct tasklet_struct *t);
```

Removes the tasklet from the list of active ones, if it's sched uled to ru n. Like *tasklet_disable*, the function may block on SMP systems waiting for the tasklet to terminate if it's currently running on another CPU.

Workqueues

```
#include <linux/workqueue.h>
struct workqueue_struct;
struct work_struct;
```

The structures representing a workqueue and a work entry, respectively.

```
struct workqueue_struct *create_workqueue(const char *name);
struct workqueue_struct *create_singlethread_workqueue(const char *name);
void destroy_workqueue(struct workqueue_struct *queue);
```

Functions for creating and destroying workqueues. A call to *create_workqueue* creates a queue with a worker thread on each processor in the system; instead, *create_singlethread_workqueue* creates a workqueue with a single worker process.

```
DECLARE_WORK(name, void (*function)(void *), void *data);
INIT_WORK(struct work_struct *work, void (*function)(void *), void *data);
PREPARE_WORK(struct work_struct *work, void (*function)(void *), void *data);
```

Macros that declare and initialize workqueue entries.

```
int queue_work(struct workqueue_struct *queue, struct work_struct *work);
int queue_delayed_work(struct workqueue_struct *queue, struct work_struct
*wo rk, unsign ed long de lay);
```

Functions that queue work for execution from a workqueue.

```
int cancel_delayed_work(struct work_struct *work);
void flush_workqueue(struct workqueue_struct *queue);
```

Use *cancel_delayed_work* to remove an entry from a workqueue; *flush_workqueue* ensures that no workqueue entries are running anywhere in the system.

```
int schedule_work(struct work_struct *work);
int schedule_delayed_work(struct work_struct *work, unsigned long delay);
void flush_scheduled_work(void);
```

Functions for working with the shared workqueue.

---

[1] The trailing *nop* instruction is required to prevent the compiler from accessing the target register in the instruction immediately following *mfc0*. This kind of interlock is typical of RISC processors, and the compiler can still schedule useful instructions in the delay slots. In this case, we use *nop* because inline assembly is a black box for the compiler and no optimization can be performed.

[2] The u in *udelay* represents the Greek letter mu and stands for *micro*.